VoltControl

Secondary Voltage Control via Model Predictive Control

Design of a Secondary Voltage Controller for the Sub-Transmission Grid

Technical Report — CentraleSupélec & RTE

Léo Guillet & Romain Fonteneau

April 28, 2026

Abstract

This report presents the design and implementation of a secondary voltage controller for a sub-transmission network with high renewable penetration, developed in partnership with RTE. The controller addresses the risk of automatic disconnection of renewable plants caused by voltage deviations at their point of connection. Three layers of actuation are coordinated: generator voltage setpoints, on-load tap changer (OLTC) positions, and shunt capacitor/reactor switching. The core of the architecture is a Model Predictive Controller (MPC) that solves a receding-horizon quadratic programme (QP) at each time step, using sensitivity matrices derived from the network Jacobian to linearise the power flow equations around the current operating point. A key design choice is to control the tap *increment* rather than the secondary voltage target given to the OLTC, because the step size is small, the tap actuator can be linearised and embedded directly into the QP as a continuous variable, enabling global co-optimisation of generator voltages and tap commands within a single convex problem. The two-step actuation delay of the tap is handled by an augmented state formulation. Shunt switching is managed by a greedy enumeration logic operating at lower priority. Limitations of the current architecture are discussed, together with perspectives for future work.

Contents

1 Introduction and Motivation

1.1 The Energy Transition and the Rise of Renewables in France

France has committed to a profound transformation of its electricity mix as part of its climate and energy policy. The *Programmation pluriannuelle de l'énergie* (PPE) and the broader European *Green Deal* set legally binding targets to reach carbon neutrality by 2050, which requires drastically reducing fossil fuel consumption across all sectors. Two parallel dynamics are reshaping the electricity system:

- **Electrification of end uses:** The decarbonisation strategy relies on replacing fossil-fuel-based processes with electrical equivalents. Heating (heat pumps), transportation (electric vehicles), and industrial processes (green hydrogen, electric furnaces) are expected to raise French electricity consumption significantly over the coming decades, reversing the stagnation observed since the 2000s. RTE's reference scenario projects an increase from approximately 550 TWh today to 650 TWh by 2035.
- **Massive deployment of variable renewables:** To meet both the carbon targets and the rising demand, France is accelerating the installation of solar photovoltaic and onshore/offshore wind capacity. The PPE3 targets between 55 and 80 GW of solar and between 50 and 55 GW of wind by 2035. Unlike the historical nuclear and hydro fleet, these sources are decentralised, intermittent, stochastic and inject power at sub-transmission level rather than at the transmission one.

This double pressure — more consumption and more variable, decentralised generation — creates unprecedented challenges for grid operators. Power flows that historically were predictable and unidirectional (from transmission toward distribution) now fluctuate rapidly and can reverse direction. Voltage profiles that were stable become volatile. Reactive power management, which was largely a transmission-level concern, now becomes critical at the 63 kV sub-transmission level where most renewable plants connect.

1.2 A Changed Grid: Technical Consequences of Renewable Integration

Conventional generators based on synchronous machines provide inherent voltage support through their excitation systems and can tolerate temporary electrical disturbances. Renewable plants connected via power electronics behave fundamentally differently:

- **Limited overload capability:** Power electronic converters are designed with strict current limits to protect semiconductor components. As a result, they cannot tolerate overcurrents beyond a narrow margin, unlike conventional generation units which can withstand temporary electrical stress.
- **Fast and strict protection relays:** Inverter-based plants disconnect almost instantaneously when voltage or current exceeds their protection thresholds. These protections are necessary to prevent damage to sensitive electronic components, leading to tripping within milliseconds.

- **Reactive power sensitivity:** High reactive power demands produce elevated currents, which — combined with the tight current limits of power electronics — quickly drive converters to their protection boundaries and trigger disconnections. In high-voltage networks, where reactive power is closely linked to voltage, voltage control becomes critical to avoid these situations.

The risk of **cascading disconnections** is therefore much higher than with a conventional fleet: a local voltage disturbance can trigger the simultaneous tripping of multiple renewable producers, leading to a sudden loss of generation that amplifies the initial disturbance and may propagate to a full blackout.

1.3 The Three Levels of Voltage Control

Voltage regulation in power systems is organised in three hierarchical control layers, operating at very different time scales and spatial scopes.

Primary control (milliseconds to seconds, local): Each generator is equipped with an Automatic Voltage Regulator (AVR) that continuously controls its terminal voltage to follow a setpoint. It reacts almost instantaneously to transient voltage drops caused by faults or sudden load changes. Primary control has no network-wide awareness: each AVR only observes its own terminal voltage and has no information about other generators or the state of the grid.

Secondary control (seconds to minutes, zonal): A supervisory controller, operating over a defined zone of the network, adjusts the voltage setpoints sent to the AVRs and coordinates reactive resources (generator reactive power, tap changers, shunt elements). Its dual role is to maintain load voltages close to their reference values and to restore the reactive power margins consumed by primary control actions. Secondary control acts much more slowly than primary control and is centralised at the zone level. **This is the level addressed in the present project.**

Tertiary control (minutes to hours, system-wide): Also called *optimal reactive-power dispatch* or voltage optimisation, tertiary control determines the optimal reactive power setpoints for the entire network, minimising losses and ensuring that each zone has sufficient reactive reserves for secondary control to operate effectively. It typically runs as an optimisation problem (Optimal Power Flow) on a time scale of 15 minutes to one hour, and its outputs serve as targets for the secondary controllers.

These three layers are complementary and operate simultaneously: primary control provides the speed, secondary control provides the zonal coordination and margin restoration, and tertiary control provides the network-wide economic optimality.

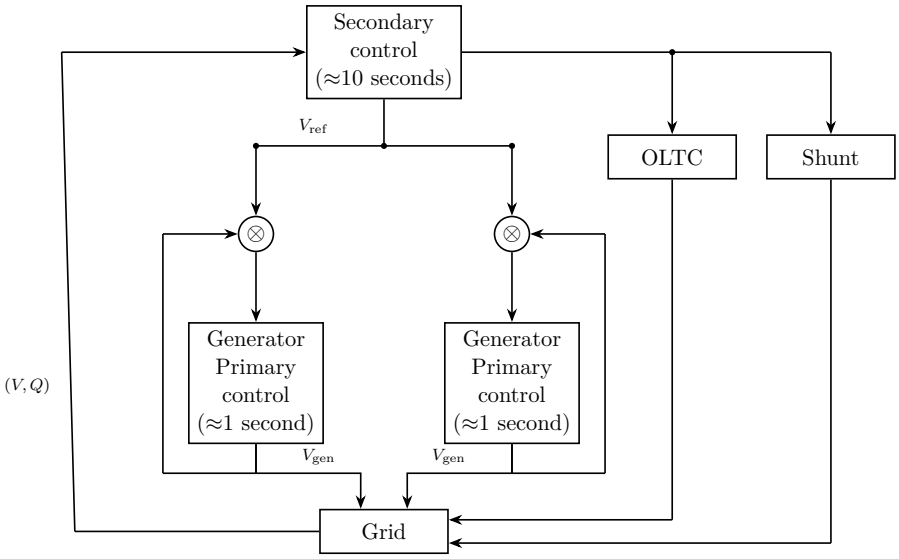


Figure 1: Primary and secondary voltage control.

1.4 The April 2025 Iberian Blackout: A Landmark Warning

On 28 April 2025, the Iberian Peninsula (Spain and Portugal) experienced a massive, near-total blackout affecting approximately 55 million people. The event unfolded in a matter of seconds: a sequence of disconnections of renewable generation plants, triggered by voltage instability in an operating context of very high renewable penetration, cascaded into a collapse of the entire synchronised zone.

The ENTSO-E investigation report, published in April 2026, identified reactive power management and voltage control as central contributing factors. Among its key recommendations directed at all European TSOs:

- **Implement or strengthen Secondary Voltage Control (SVC):** The system must be operated so that sufficient reactive power margins are always maintained on generators. Automated SVC systems should replace or supplement manual operator interventions.
- **Maintain real-time reactive reserves:** Reactive power reserves must be managed continuously to always maintain a margin capable of absorbing a sudden incident without triggering protective disconnections.
- **Integrate shunt reactors into automatic control:** Shunt reactors and capacitor banks, which were historically switched manually, should be integrated into the secondary control level and dispatched automatically to preserve margins in real time.

This event transformed what had been a theoretical engineering concern into an urgent operational priority for every TSO in Europe, including RTE.

If a generator operates continuously at its physical reactive limit $Q_{\max}$, any sudden network fault will instantly saturate its reactive capability, and the primary AVR will be unable to respond. The generator's protection relay will then trip, potentially triggering cascading failures. Secondary voltage control prevents this scenario by enforcing a *soft limit* $Q_{\text{lim}} < Q_{\max}$: generators are kept within $[-Q_{\text{lim}}, Q_{\text{lim}}]$ in normal operation, preserving a reactive power margin that primary control can exploit during fast transients.

Setting Q_{lim} correctly requires knowing the expected magnitude of disturbances — this is a design parameter of the secondary controller.

1.5 Project Objective

RTE currently lacks an automated secondary voltage control system for its subtransmission grid (63 kV). At this voltage level, the integration of renewable producers is densest and the risk of cascading disconnections is highest. This project directly responds to the operational need identified by the Iberian blackout: it investigates the feasibility of developing an automated secondary voltage controller using **Model Predictive Control (MPC)**.

The objective of this project is to design such a controller in matlab using *matpower* to simulate the electrical grid. The controller is tested on a case where load voltage references are constant, but where active and reactive load consumption and renewable active production are subject to zero-mean Gaussian white noise at each step. This noise models forecast uncertainty and acts as an unmeasured disturbance: at each step it shifts the power flow solution away from the nominal operating point, causing load voltages and generator reactive powers to deviate from their predicted trajectories. The controller has no explicit knowledge of the noise realisations and does not model them — its sensitivity matrices and QP are computed at the nominal operating point. However, it does not ignore the disturbances entirely: since the measured state $x(k)$ already reflects the cumulative effect of past noise injections, the closed-loop feedback implicitly incorporates their impact at every step.

The controller, that this project aims to design, must address the three core recommendations of the ENTSO-E report simultaneously:

- **Reactive margin preservation:** keep generator reactive power Q_{gen} away from its physical limits $[Q_{\min}, Q_{\max}]$, ensuring a margin for primary control to absorb sudden disturbances.
- **Voltage tracking:** maintain load voltages close to their reference values, preventing the voltage deviations that trigger protective disconnections.
- **Automatic shunt coordination:** integrate capacitor/reactor switching into the automatic control loop, deploying them optimally in real time rather than relying on manual operator decisions.

1.6 Why Model Predictive Control?

A naïve decentralised approach (e.g., a PI controller pairing each generator with a load bus) fails for two fundamental reasons:

1. **Coupling:** The sub-transmission grid is a strongly coupled MIMO system. Acting on one generator voltage affects all load voltages simultaneously. A SISO PI controller cannot account for this coupling and produces poor performance.
2. **Constraints:** A PI controller has no native mechanism to respect physical limits on voltages, reactive powers, or actuator increments.

MPC addresses both issues by solving at each step a constrained optimisation problem over a finite prediction horizon. It naturally handles MIMO coupling through the sensitivity model and enforces all physical constraints as hard inequality constraints in the QP.

An alternative centralised approach such as Linear Quadratic Regulation (LQR) handles the coupling but provides no mechanism to enforce physical limits on voltages, reactive powers, or actuator increments: constraint satisfaction would have to be imposed post hoc by clipping the control action, which can degrade performance or cause instability. MPC integrates these constraints directly into the optimisation, guaranteeing that every applied input is feasible by construction.

2 Network Description

2.1 Topology

The physical system is modelled using a 12-bus network derived from the IEEE 9-bus test case and implemented in MATPOWER (base: 100 MVA). It comprises three voltage levels interconnected as follows.

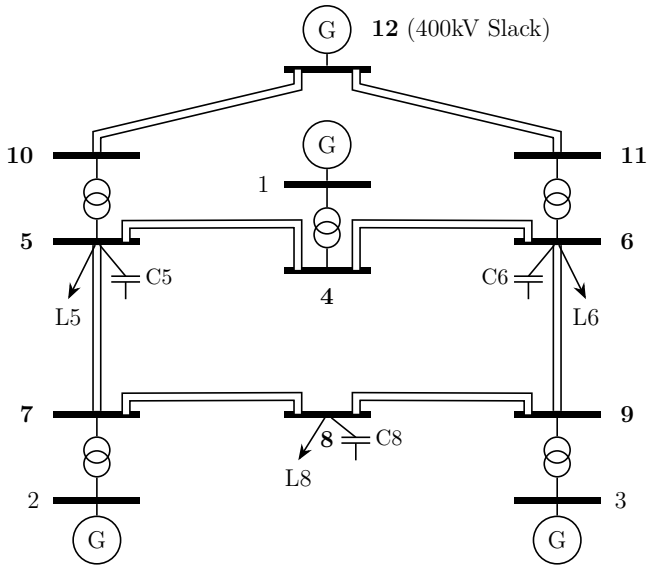


Figure 2: 12-bus sub-transmission network. Buses 10, 11, 12 are at 400 kV; buses 4–9 at 63 kV; buses 1, 2, 3 at 20 kV (generators). Double lines denote double-circuit transmission lines.

- **400 kV level:** Buses 10, 11, 12. Bus 12 hosts the slack generator (voltage-angle reference, uncontrolled by the secondary controller). The slack generator can provide almost an infinite active and reactive power.
- **63 kV sub-transmission level:** Buses 4–9. Buses 5, 6, and 8 carry active loads (L5, L6, L8) and are the *controlled buses* whose voltages must track references. These loads can be considered as the interface with the distribution grid. The Buses 4, 7, and 9 are transit buses with no load. Shunt capacitors C5, C6, C8 can be switched in or out.
- **20 kV generation level:** Buses 1, 2, 3. Three controllable PV generators (G1, G2, G3), each connected to the 63 kV network through a step-up transformer.

- **OLTC transformers:** Branches 16 (bus 10→5) and 17 (bus 11→6) are 400/63 kV transformers with On-Load Tap Changers. Their tap ratio varies in [0.9, 1.1] in steps of $step\_size = 0.00625$.
- **Other branches:** 19 fixed-ratio branches (transmission lines and step-up transformers).

The powerflow of this network will be computed with the `MATPOWER` function `runpf`.

2.2 Network Parameters

All branch impedances are given in per unit on the system base of 100 MVA. All power values are in MW or Mvar.

Loads.

Bus	P_d (MW)	Q_d (Mvar)	Fixed shunt B_s (Mvar)
5	100	30	20
6	150	40	20
8	200	50	20

Generators. The initial voltage setpoint V_g^0 is the value used at the start of the simulation before the MPC begins issuing increments.

Bus	P_g (MW)	$Q_{\max}$ (Mvar)	$Q_{\min}$ (Mvar)	V_g^0 (p.u.)	$P_{\max}$ (MW)
1	80	60	−60	1.010	100
2	70	60	−60	1.010	100
3	75	60	−60	1.010	100
12 (slack)	500	600	−600	1.0125	1000

Branches. Double-circuit lines are represented as two parallel branches with identical parameters; the table indicates the number of parallel circuits. All values are in p.u. on 100 MVA base.

Buses	r	x	b	Type	Circuits
4→1, 7→2, 9→3	0.0020	0.1500	0	Step-up trafo 20/63 kV	1 each
4–5, 6–9	0.0200	0.0950	0.0014	63 kV line	2
4–6, 5–7	0.0202	0.09595	0.001414	63 kV line	2
7–8, 8–9	0.0100	0.1000	0	63 kV line	2
10→5, 11→6	0.0020	0.1500	0	OLTC trafo 400/63 kV	1 each
10–12, 11–12	0.00047	0.00563	0.192	400 kV line	2 each

2.3 Degrees of Freedom and Control Objectives

The controller has three types of actuators, operated at different time scales and with different priorities:

1. **Generator terminal voltages** V_{g1}, V_{g2}, V_{g3} (continuous, highest priority): Applied immediately and mainly used to track load voltage references.

2. **OLTC tap positions** (quasi-continuous): Used when generator voltages alone are insufficient. The tap is mainly used to track load voltage references and increased by step of $step\_size = 0.00625$. The total value is clamped to $[0.9, 1.1]$. A tap increment decided at step k takes effect at step $k+2$: one step to store the movement, one more for it to appear in the power flow result.

3. **Shunt capacitors/reactors** at buses 5, 6, 8 (discrete, lowest priority): Mainly used to keep Q_{gen} within the soft limit Q_{lim}. Capacitors are used only if reactive power margins are too tight.

Shunt capacitors/reactors and tap changers are mechanical actuators; to limit wear, they should be activated as infrequently as possible. Tap changers take priority over shunt capacitors/reactors.

The control objective is to keep voltages V_5, V_6, V_8 close to their references while maintaining $|Q_{\text{gen},i}| \leq Q_{\text{lim}}$ for all generators, ensuring reactive power margins remain available for the primary control layer.

2.4 Actuator Implementation

The three actuator types are interfaced with the MATPOWER network model in fundamentally different ways.

Generator voltages. The voltage setpoint of each controllable generator is stored directly in the MATPOWER generator matrix `mpc.gen(:,VG)`. Applying a control action $u_k = \Delta V_{\text{gen}}$ at step k simply amounts to incrementing this field before calling the power flow. The new setpoint takes effect immediately in the next `runpf` call: there is no implementation delay for generator voltages.

Shunt capacitors and reactors. The shunt susceptance of each load bus is stored in `mpc.bus(:,BS)`. Adding a capacitor of susceptance `cap` at bus i modifies this field by `+cap`; adding a reactor modifies it by `−cap`. The modification is applied *before* the power flow call, so the effect is already present in the power flow result at step $k+1$ — there is no delay for shunt elements.

To avoid adverse interactions, a switching action at bus i is only allowed when the following eligibility conditions are satisfied simultaneously:

- The cooldown counter for bus i has elapsed (no recent switching at this bus).
- The load voltage at bus i has been stable over the last $t_{\text{cap,last}}$ steps (no ongoing transient).
- The voltage reference will not change over the next $t_{\text{cap,next}}$ steps (no imminent reference step that would immediately reverse the action).

OLTC tap changers. Unlike the two previous actuators, the tap ratio of an OLTC transformer *cannot be set directly*. The function `run_pf_oltc_step.m` implements the tap advancement logic as follows: after running the power flow with the current tap, it compares the actual secondary voltage V_{sec} to a *target secondary voltage* V_{target} provided as an input argument. If $|V_{\text{sec}} - V_{\text{target}}| > \texttt{tol}$, the tap is advanced by exactly one step ($step\_size = 0.00625$) in the direction of V_{target}, then clamped to [0.9, 1.1]. Otherwise the tap is unchanged. The secondary controller therefore communicates its intent to the OLTC not as a tap increment, but as a target voltage that induces an advance or retraction.

Because the tap advancement occurs *after* the power flow, the effect of a tap action decided at step k is not seen in the network state until step $k+2$ (one step to store the movement, one more for it to appear in the power flow result). This two-step delay is explicitly modelled by the augmented state later described.

Tap value not available. To be realistic the controller does not have direct access to the current tap ratio: `mpc.branch(TAP)` is updated internally by `run_pf_oltc_step` but is not read back by the controller (except to detect whether the tap has reached its physical limits [0.9, 1.1]).

Simplification vs. real OLTC interface. In the simulation, the target voltage V_{target} passed to `run_pf_oltc_step` can take any value. In a real substation, the OLTC control interface typically accepts only three discrete orders: raise, lower, or hold — corresponding to three canonical target voltages ($V_{\text{target}} \in \{0.95, 1.00, 1.05\}$ p.u.). The current implementation implicitly assumes a richer interface; bridging this gap is a practical consideration for a real deployment.

3 Linearised State-Space Model

3.1 Power Flow and Jacobian

The power flow equations for an n-bus network express the active and reactive power injections as nonlinear functions of bus voltage magnitudes V and angles θ. Linearised about a given operating point, they give:

$$\begin{pmatrix}\Delta P\\ \Delta Q\end{pmatrix} = JAC \begin{pmatrix}\Delta\theta\\ \Delta V\end{pmatrix}, \qquad JAC = \begin{pmatrix}\frac{\partial P}{\partial\theta} & \frac{\partial P}{\partial V}\\ \frac{\partial Q}{\partial\theta} & \frac{\partial Q}{\partial V}\end{pmatrix}.$$

Why reactive power is controlled by voltage in high-voltage networks. In high-voltage transmission and sub-transmission networks, branches are predominantly inductive: the reactance X is much larger than the resistance R. For the 63 kV lines of this network $X/R \approx 4.8$, and for the 400 kV lines $X/R \approx 12$. In the limit $R \to 0$, the reactive power injected at a bus i connected to bus j through a reactance X_{ij} is:

$$Q_i \approx \frac{V_i^2 - V_i V_j \cos\theta_{ij}}{X_{ij}} \approx \frac{V_i(V_i - V_j)}{X_{ij}},$$

which depends directly on the *voltage magnitude difference* $(V_i - V_j)$ and only weakly on the angle θ_{ij} (since $\cos\theta_{ij} \approx 1$ for small angles). Conversely, active power is primarily governed by the angle difference θ_{ij}. This **decoupling between P-θ and Q-V** is the

standard justification for the fast-decoupled power flow and implies that, in high-voltage networks, the most effective lever to control reactive power (and hence bus voltages) is to act on voltage magnitudes. The cross-coupling terms ($\partial Q/\partial\theta$ and $\partial P/\partial V$) are small compared to the diagonal blocks, so:

$$\Delta Q \approx JAC_{QV}\,\Delta V,$$

where $JAC_{QV} = \partial Q/\partial V$ is the bottom-right block of the full Jacobian. This is the only block needed to derive all sensitivity matrices used in the controller.

Need for a linearised model. The MPC, that will be designed, must predict how the controlled outputs (V_{load}, Q_{gen}) will evolve as a function of the control inputs, in order to formulate a Quadratic Program (QP). The exact power flow equations are nonlinear and solved iteratively by MATPOWER (`runpf`). The sensitivity matrices, computed below, derived from the linearised Jacobian at the current operating point, replace these nonlinear maps by affine approximations. This reduces the MPC prediction to a simple matrix-vector product, making the QP both tractable and solvable in real time.

Key assumption: Jacobian availability. In this simulation, the Jacobian JAC is recomputed analytically by MATPOWER's `makeJac` at the end of every control step by default (parameter `t_jac` = 0, corresponding to a 10 s refresh rate); it can also be recomputed every n steps by setting `t_jac` = n. In a real deployment, the jacobian is typically refreshed every **5 minutes** and could be significantly stale between refreshes. This is an important idealisation of the present work: the controller benefits from a continuously up-to-date operating-point description that would not be available in practice. The impact of a stale Jacobian on performance and constraint satisfaction is identified as a direction for future work. The Jacobian depends on the current network topology (connected shunt elements, tap positions, voltage setpoints), so it must be recomputed whenever the topology changes.

3.2 Sensitivity Matrices with Respect to Generator Voltages

The sub-transmission buses are partitioned into two sets:

- **Load buses (PQ):** buses 4–9, where reactive power injection is imposed by the load model. The reactive power balance at these buses is $\Delta Q_l = 0$ (the load absorbs whatever the network supplies).
- **Generator buses (PV):** buses 1, 2, 3, where the voltage magnitude is controlled.

From the reactive power balance at load buses:

$$0 = \Delta Q_l = \frac{\partial Q_l}{\partial V_l}\,\Delta V_l + \frac{\partial Q_l}{\partial V_g}\,\Delta V_g,$$

which gives the load-voltage sensitivity to generator voltages:

$$\boxed{C_v = \frac{\Delta V_l}{\Delta V_g} = -\left(\frac{\partial Q_l}{\partial V_l}\right)^{-1}\frac{\partial Q_l}{\partial V_g}} \tag{1}$$

The reactive power at generator buses depends on both the load voltages (which respond to ΔV_g via C_v) and on V_g directly:

$$\Delta Q_g = \frac{\partial Q_g}{\partial V_l}\,\Delta V_l + \frac{\partial Q_g}{\partial V_g}\,\Delta V_g = \left(\frac{\partial Q_g}{\partial V_l} C_v + \frac{\partial Q_g}{\partial V_g}\right) \Delta V_g,$$

yielding the generator-reactive-power sensitivity:

$$\boxed{C_q = \frac{\Delta Q_g}{\Delta V_g} = \frac{\partial Q_g}{\partial V_l}\,C_v + \frac{\partial Q_g}{\partial V_g}} \tag{2}$$

3.3 Sensitivity Matrices with Respect to Shunt Capacitors

When a shunt capacitor of susceptance B_{sh} is connected at load bus i, it injects reactive power $Q_c = B_{\text{sh}} \cdot V_i^2$. Since the generator setpoints are held constant during a capacitor switching ($\Delta V_g = 0$), the reactive balance at load buses becomes:

$$0 = \Delta Q_l = -B_{\text{sh}} V_l^2\, e_i + \frac{\partial Q_l}{\partial V_l}\,\Delta V_l,$$

where e_i is the standard basis vector for bus i. Solving for ΔV_l per unit of B_{sh}:

$$\boxed{C_{v,\text{cap}} = \frac{\Delta V_l}{B_{\text{sh}}} = \left(\frac{\partial Q_l}{\partial V_l}\right)^{-1} V_l^2} \tag{3}$$

The corresponding sensitivity for generator reactive power:

$$\boxed{C_{q,\text{cap}} = \frac{\Delta Q_g}{B_{\text{sh}}} = \frac{\partial Q_g}{\partial V_l}\left(\frac{\partial Q_l}{\partial V_l}\right)^{-1} V_l^2} \tag{4}$$

3.4 Sensitivity Matrices with Respect to OLTC Tap Increments

An increment *step_size* of the tap ratio of an OLTC transformer connected at a secondary bus s injects reactive power $Q_{\text{sec}} \approx V_s^2/X \cdot \mathit{step\_size}$ at that bus, where X is the transformer leakage reactance. The derivation assumes that the **primary (400 kV) side voltage is held constant** during the tap increment: the 400 kV network is much stiffer than the 63 kV network (lower impedances, large slack generator), so a tap movement on one 400/63 kV transformer has negligible effect on the 400 kV bus voltages. The reactive balance at secondary load buses gives:

$$\boxed{C_{v,\text{tap}} = \frac{\Delta V_l}{\mathit{step\_size}} = \left(\frac{\partial Q_l}{\partial V_l}\right)^{-1} \frac{V_l^2}{X}} \tag{5}$$

$$\boxed{C_{q,\text{tap}} = \frac{\Delta Q_g}{\mathit{step\_size}} = \frac{\partial Q_g}{\partial V_l}\left(\frac{\partial Q_l}{\partial V_l}\right)^{-1} \frac{V_l^2}{X}} \tag{6}$$

In practice, V_l^2/X is evaluated at the current operating point using the bus voltage from the last power flow and the transformer parameters.

Important distinction: $C_{v,\text{tap}}$ relates voltage changes to *tap increments*, not to the target secondary voltage V_{target}. A linearised relationship with V_{target} does not exist (the relation is strongly nonlinear), which is why the MPC will use the tap increment as its control variable for the OLTC and not V_{target}.

3.5 Validation of the linear approximation.

The sensitivity matrices derived analytically from the Jacobian have been cross-validated against empirical sensitivity matrices obtained by running a full nonlinear power flow with a perturbation of the same magnitude as the actual control actions applied in simulation (generator voltage increments up to $u_{\max}$, one tap step, one capacitor switching). Over the operating range explored in the simulations, the element-wise relative error between the analytical and empirical matrices is at most **15 %**. This confirms that the linearisation is sufficiently accurate to be used inside the MPC prediction model, and that the affine approximation does not introduce errors large enough to compromise constraint satisfaction or voltage tracking performance.

3.6 State, Input, and Output Vectors

The MPC operates on a discrete-time linear model. The state vector captures all quantities relevant for control and constraint enforcement:

$$x(k) = \begin{pmatrix} V_{\text{load}}(k) \\ Q_{\text{gen}}(k) \\ V_{\text{gen}}(k) \end{pmatrix} \in \mathbb{R}^{n_l+n_g+n_g}$$

where $n_l = 6$ is the number of load buses (buses 4–9) and $n_g = 3$ is the number of controllable generators. Generator voltages are included in the state to enforce their bounds as hard constraints.

The control input is the increment of generator voltage setpoints:

$$u(k) = \Delta V_{\text{gen}}(k) \in \mathbb{R}^{n_g}, \qquad V_{\text{gen}}(k+1) = V_{\text{gen}}(k) + \Delta V_{\text{gen}}(k).$$

The output vector contains the quantities penalised in the cost function:

$$y(k) = \begin{pmatrix} V_{\text{load,target}}(k) \\ Q_{\text{gen}}(k) \end{pmatrix} \in \mathbb{R}^{n_t+n_g}$$

where $n_t = 3$ is the number of target load buses (buses 5, 6, 8 — the only buses with active loads).

3.7 Linearised Dynamics

From the sensitivity matrices (1) and (2), the discrete-time dynamics of the state are:

$$\begin{cases} V_{\text{load}}(k{+}1) = V_{\text{load}}(k) + C_v(k)\,\Delta V_{\text{gen}}(k) \\ Q_{\text{gen}}(k{+}1) = Q_{\text{gen}}(k) + C_q(k)\,\Delta V_{\text{gen}}(k) \\ V_{\text{gen}}(k{+}1) = V_{\text{gen}}(k) + \Delta V_{\text{gen}}(k) \end{cases}$$

This gives the standard linear state-space form $x(k+1) = A\,x(k) + B(k)\,u(k)$, $y(k) = C\,x(k)$, with:

$$A = I_{n_l+2n_g}, \qquad B(k) = \begin{pmatrix} C_v(k) \\ C_q(k) \\ I_{n_g} \end{pmatrix}, \qquad C = \begin{pmatrix} I_{n_t} & 0 & 0 \\ 0 & I_{n_g} & 0 \end{pmatrix}. \tag{7}$$

The matrix $A = I$ captures the key assumption: without a control action, the linearised model predicts that voltages and reactive powers remain constant. This is justified by the slow time scale of secondary voltage control. The matrix $B(k)$ is recomputed at every step from the current $JAC(k)$, providing a time-varying linearisation that tracks the operating point.

4 MPC Formulation

4.1 Stacked Vectors Over the Prediction Horizon

The MPC optimises over a prediction horizon of N steps. Define the stacked state, input, and output vectors:

$$\underline{U}(k) = \begin{pmatrix} u(k) \\ \vdots \\ u(k{+}N{-}1) \end{pmatrix}, \quad \underline{Y}(k) = \begin{pmatrix} y(k{+}1) \\ \vdots \\ y(k{+}N) \end{pmatrix}, \quad \underline{Y}_c(k) = \begin{pmatrix} y_c(k{+}1) \\ \vdots \\ y_c(k{+}N) \end{pmatrix},$$

where $y_c(k) = (V_{\text{ref}}(k);\, 0)$ is the reference output (voltage reference for load buses; zero reactive power as the nominal target). Since $A = I$, the predicted output trajectory is:

$$\underline{Y}(k) = \tilde{C}\,\Pi\,x(k) + \tilde{C}\,\Gamma\,\underline{U}(k),$$

where $\tilde{C} = \text{diag}(C, \ldots, C) \in \mathbb{R}^{N(n_t+n_g)\times N(n_l+2n_g)}$, and the prediction matrices are:

$$\Pi = \begin{pmatrix} I \\ \vdots \\ I \end{pmatrix} \in \mathbb{R}^{N(n_l+2n_g)\times(n_l+2n_g)}, \qquad \Gamma = \begin{pmatrix} B & 0 & \cdots \\ B & B & \cdots \\ \vdots & & \ddots \end{pmatrix} \in \mathbb{R}^{N(n_l+2n_g)\times Nn_g}.$$

Since $A = I$, all rows of Π are identical, and the first row of each block row of Γ is B while all sub-diagonal blocks are also B (each past input contributes to all future states).

4.2 Cost Function

The MPC minimises the sum of squared tracking errors and reactive power deviations over the horizon:

$$J = \sum_{i=1}^{N} \left[\|V_{\text{load,target}}(k{+}i) - V_{\text{ref}}\|^2 + \alpha\, \|Q_{\text{gen}}(k{+}i)\|^2 \right] \tag{8}$$

with $\alpha = 0.001$. Voltages and reactive powers would be weighted equally if $\alpha = 1$; with $\alpha = 0.001$ the voltage tracking term (coefficient 1) is dominant; the reactive power term ($\alpha \ll 1$) acts as a soft regularisation, biasing the optimal solution away from reactive extremes without preventing the controller from tracking the voltage reference.

In matrix form, the cost is:

$$J = \|\underline{Y}(k) - \underline{Y}_c(k)\|_\Psi^2 = (\underline{Y} - \underline{Y}_c)^\top \Psi (\underline{Y} - \underline{Y}_c),$$

where $\Psi = \text{diag}(Q_w, \ldots, Q_w) \in \mathbb{R}^{N(n_t+n_g)\times N(n_t+n_g)}$ and $Q_w = \text{diag}(1, \ldots, 1,\, \alpha, \ldots, \alpha)$ is the per-step weighting matrix (weight 1 on V_{load} errors, weight α on Q_{gen}).

4.3 QP Reformulation

Substituting the predicted output $\underline{Y} = \tilde{C}\Pi\, x + \tilde{C}\Gamma\, \underline{U}$ into (8) and expanding:

$$J = \frac{1}{2}\,\underline{U}^\top H\, \underline{U} + (f_1\, x(k) - f_2\, \underline{Y}_c(k))^\top \underline{U} + \text{const},$$

where the quadratic and linear terms of the QP are:

$$H = \Gamma^\top \tilde{C}^\top \Psi \tilde{C} \Gamma \tag{9}$$

$$f_1 = \Gamma^\top \tilde{C}^\top \Psi \tilde{C} \Pi \tag{10}$$

$$f_2 = \Gamma^\top \tilde{C}^\top \Psi \tag{11}$$

The constant term (independent of $\underline{U}$) does not affect the optimal solution and is omitted from the QP. The full gradient vector passed to `quadprog` is therefore $f = f_1 x(k) - f_2 \underline{Y}_c(k)$.

Note that H is computed once at initialisation from the constant parts of the problem in `cal_mat_cst.m`, while f_1 depends on Π and hence on $B(k)$, so it is recomputed at every step in `Cal_mat_var.m`.

4.4 Constraints

The QP enforces the following hard inequality constraints:

1. **Actuator increment bounds:**

$$|\Delta V_{\text{gen}}(k{+}i)| \le u_{\max}, \quad i = 0, \ldots, N{-}1$$

2. **Load voltage bounds:**

$$V_{l,\min} \le V_{\text{load}}(k{+}i) \le V_{l,\max}, \quad i = 1, \ldots, N$$

3. **Generator reactive power bounds:**

$$Q_{\min} \le Q_{\text{gen}}(k{+}i) \le Q_{\max}, \quad i = 1, \ldots, N$$

4. **Generator terminal voltage bounds:**

$$V_{g,\min} \le V_{\text{gen}}(k{+}i) \le V_{g,\max}, \quad i = 1, \ldots, N$$

All these constraints are linear in $\underline{U}$ and can be written in compact form as:

$$A_{\text{ineq}}\, \underline{U} \le b_1 - b_2\, x(k),$$

where b_1 collects the constant parts of the right-hand sides (bounds), and $b_2\, x(k)$ accounts for the dependence on the initial state (the prediction matrices Π, Γ express future states as functions of $x(k)$ and $\underline{U}$). The constraint matrix A_{ineq} and the constant vector b_1 are assembled in `cal_mat_cst.m`; the state-dependent vector $b_2\, x(k)$ is built at each step in `Cal_mat_var.m` because it depends on $B(k)$ through Π.

The QP is solved by MATLAB's `quadprog`:

$$\underline{U}^\star = \arg\min_{\underline{U}} \; \frac{1}{2}\underline{U}^\top H \underline{U} + f^\top \underline{U} \quad \text{subject to} \quad A_{\text{ineq}}\, \underline{U} \le b_1 - b_2\, x(k). \tag{12}$$

The receding horizon principle is applied: only the first block $u_k^\star = \underline{U}^\star(1 : n_g)$ is applied to the generators at step k.

5 Shunt Capacitor/Reactor Logic

5.1 Motivation

The MPC cost (8) includes a soft penalty $\alpha\|Q_{\text{gen}}\|^2$ on reactive power, but with $\alpha \ll 1$ this term is too weak to enforce the soft limit Q_{lim} as a hard constraint. Since shunt switching is a coarse binary decision (add or remove one element per bus), the quasi-continuous approximation used for OLTC taps does not apply here. A separate enumeration-based logic (implemented in `add_cap.m`) therefore operates after the MPC, with lower priority. It is triggered only when at least one generator is predicted to exceed Q_{lim}, and selects the single best switching action.

5.2 Eligibility Conditions

A bus i is eligible for a switching action at step k if and only if:

1. Its cooldown counter has expired: $\texttt{cap\_availability}(i) = 0$. (There is a delay between two switching actions.)

2. The load voltage at bus i has been *stable* over the last $t_{\text{cap,last}}$ steps:

$$\max_{j=1}^{t_{\text{cap,last}}} |V_i(k-j+1) - V_i(k-j)| \leq \texttt{var\_v\_cap\_last}.$$

3. The voltage reference will not change over the next $t_{\text{cap,next}}$ steps:

$$\max_{j=0}^{t_{\text{cap,next}}-1} |V_{\text{ref}}(k+j+1) - V_{\text{ref}}(k+j)| \leq \texttt{var\_v\_cap\_next}.$$

These conditions prevent switching during transients and when the reference is about to change, which could otherwise trigger immediately-reversed actions and increase tap wear.

5.3 Criterion and Algorithm

The criterion for the capacitor logic is the sum of squared reactive power deviations outside the soft limit:

$$J_{\text{cap}} = \sum_{j:\,|Q_j|>Q_{\text{lim}}} \hat{Q}_j(k{+}1)^2.$$

The algorithm adds or removes at most one capacitor or reactor per step. To do that it enumerates all $2n_t + 1$ candidate actions (add capacitor or reactor on each eligible bus, or do nothing). For each candidate action $u_{\text{cap}}(i) \in \{+1, -1\}$ (where $u_{\text{cap}} = 0$ means no action):

1. Predict the state at $k+1$ and using the sensitivity matrices:

$$\hat{V}_l(k{+}1) = \hat{V}_l^{\text{MPC}}(k{+}1) + u_{\text{cap}}(i)\,\texttt{cap}\,C_{v,\text{cap}}(i)$$
$$\hat{Q}_g(k{+}1) = \hat{Q}_g^{\text{MPC}}(k{+}1) + u_{\text{cap}}(i)\,\texttt{cap}\,C_{q,\text{cap}}(i)$$

   where $\hat{x}^{\text{MPC}}(k{+}1) = Ax(k) + Bu(k)$ is the state predicted using the control signal given by the MPC (before the capacitor action) and the linearized model. A predicted state is used to ensure that the logic takes into account the impact of the MPC.

2. Check hard constraints at both $k+1$: $V_{l,\min} \leq \hat{V}_l \leq V_{l,\max}$ and $Q_{\min} \leq \hat{Q}_g \leq Q_{\max}$.

3. Reject the action if any hard constraint is violated.

4. Accept if J_{cap} is strictly decreased. Update J_{cap}.

The winning action is applied by modifying `mpc.bus(:,BS)` (shunt susceptance) before the power flow. Since the modification takes place before `runpf`, the capacitor effect is already present in the power flow result and therefore in $x(k+1)$ — there is no delay, unlike the OLTC.

The cooldown counter for the switched bus is reset to t_{cap} and all other counters are decremented.

6 Augmented MPC with Integrated OLTC

6.1 Limitations of the Naive Tap Strategy

A first natural approach sets $V_{\text{target,secondary}}(j) = V_{\text{ref}}(j)$ for each OLTC transformer j, letting `run_pf_oltc_step` advance the tap locally whenever the secondary voltage deviates from its reference. This is locally optimal for each individual transformer but suffers from several drawbacks:

- The OLTC ignores Q_{gen}: the tap may move even when a generator is already near its reactive limit.
- There is no coordination with the MPC: the MPC and the OLTC act independently on the same buses, potentially conflicting.
- The two-step delay of the tap is not modelled, so the controller cannot anticipate its effect.

The final architecture integrates the OLTC commands directly into the MPC as additional optimisation variables, enabling global co-optimisation with the generator voltage setpoints.

6.2 Two-Step Tap Delay

The function `run_pf_oltc_step.m` implements the power flow and tap advancement in the following order:

1. Run the power flow with the current tap and generator setpoints.
2. For each OLTC j: if $|V_{\text{sec},j} - V_{\text{target},j}| > \texttt{tol}$, advance the tap by one step toward the target.
3. Clamp the resulting tap to $[0.9, 1.1]$.

Because the tap is modified *after* the power flow, a tap increment decided at step k does not appear in the power flow result until step $k+2$:

$$k \xrightarrow{\text{decide}} k+1 \xrightarrow{\text{tap stored}} k+2 \xrightarrow{\text{effective}} .$$

This two-step delay must be explicitly modelled for the MPC to issue correct anticipatory commands.

6.3 Augmented State and Extended Input

The in-flight OLTC command $u_{\text{OLTC}}(k-1)$ (decided at the previous step, not yet effective) is appended to the state to form the augmented state vector:

$$\tilde{x}(k) = \begin{pmatrix} x(k) \\ u_{\text{OLTC}}(k-1) \end{pmatrix} \in \mathbb{R}^{n_l+2n_g+n_{\text{OLTC}}}, \tag{13}$$

where $n_{\text{OLTC}} = 2$ is the number of OLTC transformers. The extended input now jointly optimises generator voltages and tap increments:

$$U(k) = \begin{pmatrix} \Delta V_{\text{gen}}(k) \\ u_{\text{OLTC}}(k) \end{pmatrix} \in \mathbb{R}^{n_g+n_{\text{OLTC}}}. \tag{14}$$

6.4 Augmented Dynamics

The tap increment $u_{\text{OLTC}}(k)$ decided at step k is stored in the augmented state at $k+1$, and acts on the physical state x at step $k+2$ via the sensitivity matrix $B_{\text{OLTC}}(k) = (C_{v,\text{tap}};\, C_{q,\text{tap}};\, 0)$ (the tap does not directly affect V_{gen}). This gives the augmented dynamics:

$$\tilde{x}(k+1) = \tilde{A}(k)\,\tilde{x}(k) + \tilde{B}(k)\,U(k), \tag{15}$$

with:

$$\tilde{A}(k) = \begin{pmatrix} I & B_{\text{OLTC}}(k) \\ 0 & 0 \end{pmatrix}, \qquad \tilde{B}(k) = \begin{pmatrix} B(k) & 0 \\ 0 & I_{n_{\text{OLTC}}} \end{pmatrix}. \tag{16}$$

The delay mechanism works as follows:

- At step k: $u_{\text{OLTC}}(k)$ is stored in $\tilde{x}(k+1)$ via the bottom-right identity block of $\tilde{B}$.
- At step $k+1$: the block B_{OLTC} of $\tilde{A}$ propagates $u_{\text{OLTC}}(k)$ into the physical state x, making the tap effective at $k+2$.

No explicit state correction is needed: the delay is captured entirely within the augmented model.

Key property: $\tilde{A}^n = \tilde{A}$ for all $n \geq 1$. This can be verified by direct computation: $\tilde{A}^2 = \tilde{A}\cdot\tilde{A}$, and since the bottom row of $\tilde{A}$ is zero, this equals $\tilde{A}$ (by induction). This closed-form property allows Π and Γ to be computed analytically without matrix exponentiation:

$$\Pi = \text{repmat}(\tilde{A}, N, 1) \in \mathbb{R}^{N(n_l+2n_g+n_{\text{OLTC}})\times(n_l+2n_g+n_{\text{OLTC}})} \tag{17}$$

$$\Gamma_{ij} = \begin{cases} \tilde{B} & \text{if } i = j \\ \tilde{A}\tilde{B} & \text{if } i > j \end{cases} \tag{18}$$

where $\Gamma \in \mathbb{R}^{N(n_l+2n_g+n_{\text{OLTC}})\times N(n_g+n_{\text{OLTC}})}$.

6.5 Updated Cost Function

The cost function for the augmented MPC is:

$$J = \sum_{i=1}^{N} \left[\|V_{\text{load,target}}(k+i) - V_{\text{ref}}\|^2 + \alpha\, \|Q_{\text{gen}}(k+i)\|^2 + \beta\, \|u_{\text{OLTC}}(k+i)\|^2 \right] \tag{19}$$

with $\alpha = 0.001$ (weight on Q_{gen}) and $\beta = 1$ (weight on OLTC magnitude). The three terms serve distinct roles:

- $\|V_{\text{load}} - V_{\text{ref}}\|^2$ (coefficient 1): primary objective — track the voltage reference.
- $\alpha \|Q_{\text{gen}}\|^2$: secondary regularisation — prevent reactive power from building up.
- $\beta \|u_{\text{OLTC}}\|^2$: OLTC usage cost — the tap is only activated when generator voltages alone cannot maintain the reference. This replaces the former ad hoc priority logic.

The output matrix C and the weighting matrix Ψ are defined on the augmented state, with the u_{OLTC} columns of C set to zero (the last n_{OLTC} components of $\tilde{x}$ are not tracked):

$$Q_w = \text{diag}\left(\underbrace{1, \ldots, 1}_{n_t}, \underbrace{\alpha, \ldots, \alpha}_{n_g}\right), \qquad \Psi = \text{diag}(Q_w, \ldots, Q_w) \in \mathbb{R}^{N(n_t+n_g)\times N(n_t+n_g)}.$$

The OLTC magnitude penalty is added to the QP Hessian as:

$$H_\gamma = \frac{\beta}{\alpha} S_{\text{OLTC}}^\top S_{\text{OLTC}}, \tag{20}$$

where $S_{\text{OLTC}} = \text{diag}([0_{n_g}, I_{n_{\text{OLTC}}}], \ldots) \in \mathbb{R}^{Nn_{\text{OLTC}}\times N(n_g+n_{\text{OLTC}})}$ selects the OLTC component from the stacked input vector $\underline{U}$.

Remark on numerical conditioning. The absolute values of the weights ($\alpha = 0.001$, $\beta = 1$) would give a Hessian with very small eigenvalues, causing numerical convergence issues in `quadprog`. Multiplying all weights by $1/\alpha = 1000$ yields an equivalent problem with better-conditioned matrices while keeping the same optimal $\underline{U}^\star$. The implementation in `cal_mat_cst.m` uses the normalised weights:

$$Q_w = \text{diag}\left(\underbrace{\frac{1}{\alpha}, \ldots,}_{n_t} \underbrace{1, \ldots}_{n_g}\right), \qquad H_\gamma = \frac{\beta}{\alpha} S_{\text{OLTC}}^\top S_{\text{OLTC}}.$$

6.6 Updated QP Formulation

The augmented QP has the same structure as (12) but with the augmented state $\tilde{x}$, the extended input U, and the augmented matrices:

$$H = \Gamma^\top \tilde{C}^\top \Psi \tilde{C} \Gamma + H_\gamma \tag{21}$$

$$f_1 = \Gamma^\top \tilde{C}^\top \Psi \tilde{C} \Pi \tag{22}$$

$$f_2 = \Gamma^\top \tilde{C}^\top \Psi \tag{23}$$

The gradient passed to `quadprog` is $f = f_1\, \tilde{x}(k) - f_2\, \underline{Y}_c(k)$.

The constraint matrix now has dimensions compatible with the extended input $U \in \mathbb{R}^{N(n_g+n_{\text{OLTC}})}$:

$$A_{\text{ineq}}\, \underline{U} \leq b_1 - b_2\, \tilde{x}(k),$$

where:

$$A_{\text{ineq}} = \begin{pmatrix} H_u \\ H_x\Gamma \end{pmatrix}, \qquad b_2 = \begin{pmatrix} 0 \\ H_x\Pi \end{pmatrix}.$$

H_u enforces actuator bounds ($|\Delta V_{\text{gen}}| \leq u_{\max}$, $|u_{\text{OLTC}}| \leq step\_size$). H_x selects V_{load}, Q_{gen}, V_{gen} from the predicted augmented state for state constraint enforcement.

Symmetry enforcement: After computing H, the matrix is symmetrised as $H \leftarrow (H + H^\top)/2$ to eliminate numerical asymmetry from floating-point arithmetic, which `quadprog` requires.

6.7 Tap Discretisation

The QP returns a continuous OLTC command $u^{\star}_{\mathrm{OLTC},k}(j) \in [-step\_size, step\_size]$ for each transformer j. This is converted to a physical (discrete) tap request by thresholding at $\varepsilon = step\_size/2$:

$$u_{\mathrm{OLTC},k}(j) = \begin{cases} +step\_size & \text{if } u^{\star}_{\mathrm{OLTC},k}(j) > \varepsilon \\ -step\_size & \text{if } u^{\star}_{\mathrm{OLTC},k}(j) < -\varepsilon \\ 0 & \text{otherwise (no movement)} \end{cases}$$

The direction is then encoded into a *target secondary voltage* that is passed to `run_pf_oltc_step`:

$$u_{\mathrm{OLTC},k}(j) > 0 \;\Rightarrow\; V_{\mathrm{target},j} = V_{\mathrm{sec},j} - 2\,\mathtt{tol} \quad \text{(request tap increase)}$$

$$u_{\mathrm{OLTC},k}(j) < 0 \;\Rightarrow\; V_{\mathrm{target},j} = V_{\mathrm{sec},j} + 2\,\mathtt{tol} \quad \text{(request tap decrease)}$$

$$u_{\mathrm{OLTC},k}(j) = 0 \;\Rightarrow\; V_{\mathrm{target},j} = V_{\mathrm{sec},j} \quad \text{(no movement)}$$

The function `run_pf_oltc_step` then advances the tap by *step_size* toward the target if the difference exceeds `tol`, or holds if within the deadband.

The discrete command $u_{\mathrm{OLTC},k}(j)$ is stored as $u_{\mathrm{OLTC,prev}}$ and placed in the augmented state at the next step.

6.8 Dynamic Constraints

At each step, additional constraints are added to the QP based on the current state of the OLTC and the network:

1. **Cooldown block:** After a tap action on transformer j, an availability counter `oltc_availability`(j) is set to a cooldown duration. For all subsequent steps while the cooldown is active, the constraint $u_{\mathrm{OLTC}}(j) = 0$ is enforced at every horizon step by injecting equality constraints (implemented as two opposing inequalities $\pm u_{\mathrm{OLTC}}(j) \leq 0$).

2. **Transient block:** If the secondary bus voltage has been unstable over the last $t_{\mathrm{OLTC,last}}$ steps, the tap is blocked ($u_{\mathrm{OLTC}}(j) = 0$) for the entire horizon, preventing tap action during transients.

3. **Reference stability check:** If the voltage reference is predicted to change within the next $t_{\mathrm{OLTC,next}}$ steps, the tap is similarly blocked.

4. **Physical limit:** If the tap has reached its maximum $a_{\max} = 1.1$, a unilateral constraint $u_{\mathrm{OLTC}}(j) \leq 0$ prevents further increase. Symmetrically, at $a_{\min} = 0.9$, the constraint $u_{\mathrm{OLTC}}(j) \geq 0$ prevents further decrease.

These dynamic constraints are assembled in `main.m` at each step and appended to A_{ineq}, b_{full} before calling `quadprog`.

6.9 Interaction Between OLTC Delay and Shunt Logic

Because the tap decided at step k only becomes effective at step $k{+}2$, the shunt switching logic must evaluate reactive power margins at step $k{+}2$ rather than $k{+}1$. Evaluating at $k{+}1$ would ignore the incoming tap effect: the logic might add a capacitor to compensate for an excess of reactive power, only to reverse that action one step later once the tap effect appears — producing unnecessary switching oscillations. To avoid this, the predicted state $\hat{x}(k{+}2)$ is used as the primary criterion for selecting a switching action. However, $\hat{x}(k{+}1)$ is also computed and checked: the eligibility test verifies that the selected action does not violate hard voltage or reactive power bounds at *either* $k{+}1$ or $k{+}2$, ensuring constraint satisfaction at both horizons.

7 Complete Control Loop

7.1 Key Implementation Decisions

Before detailing the per-step sequence, the following implementation choices shape the overall structure of the controller.

Centralised Jacobian. A single Jacobian *JAC* is computed once per step and shared by all sensitivity functions. This avoids redundant power flow calls and ensures consistency between C_v, C_q, $C_{v,\text{cap}}$, $C_{q,\text{tap}}$.

Separation of constant and variable matrices. The matrices that do not depend on the operating point (Ψ, H_γ, H_u, H_x, b_1) are computed once in `cal_mat_cst.m`. The operating-point-dependent matrices (which depend on $B(k)$ through $C_v(k)$ and $C_q(k)$) are recomputed at every step in `Cal_mat_var.m`, minimising redundant computation.

Augmented state as delay model. The two-step tap delay is captured by the augmented state (13) rather than by an explicit observer or delay buffer. This approach keeps the MPC problem a standard QP with no additional structure, and the closed-form $\tilde{A}^n = \tilde{A}$ property simplifies the computation of Π and Γ.

Discrete actuators and the QP. Discrete control variables cannot be included directly in the QP optimisation: doing so would require Mixed-Integer Quadratic Programming (MIQP), which introduces branch-and-bound search with unpredictable solve times, prevents the use of convex solvers such as `quadprog`, and no longer guarantees finding the global optimum or certifying infeasibility within the sampling period. Two strategies are used depending on the granularity of the discrete variable. For OLTC taps, the step size is 0.00625 p.u. over a range of [0.9, 1.1], giving 33 discrete levels — fine enough that the tap variable is treated as continuous in the QP and the resulting command is simply thresholded at $\pm\varepsilon = \textit{step\_size}/2$ after the solve, with negligible suboptimality. For shunt capacitors and reactors, only one element per bus can be switched at a time (a coarse binary decision), so the quasi-continuous approximation is not valid; a separate enumeration logic (`add_cap.m`) is used instead, as described in Section 5.

7.2 Initialisation

Before the simulation loop, the following operations are performed once:

1. Load the network (`case9xx_Bsh.m`) and run an initial power flow to compute $x(0)$.
2. Compute the initial Jacobian JAC with `makeJac`.
3. Build constant MPC matrices ($C, \Psi, \tilde{C}, H_u, H_x, b_1, H_\gamma$) with `cal_mat_cst.m`. These matrices depend only on network dimensions and weight parameters, not on the operating point.

7.3 Per-Step Control Sequence

At each step k, the controller executes the following operations in order:

1. **Sensitivity matrices:** Compute $C_v(k)$, $C_q(k)$, $C_{v,\text{tap}}(k)$, $C_{q,\text{tap}}(k)$ from $JAC(k)$ using `Cal_CvCq.m` and `Cal_Cv_tapCq_tap.m`.
2. **Variable MPC matrices:** Build $\tilde{A}(k)$, $\tilde{B}(k)$, $\Pi(k)$, $\Gamma(k)$, $H(k)$, $f_1(k)$, $f_2(k)$, $A_{\text{ineq}}(k)$, $b_2(k)$ using `Cal_mat_var.m`.
3. **Augmented state:** Form $\tilde{x}(k) = (x(k);\ u_{\text{OLTC,prev}})$.
4. **Dynamic constraints:** Append cooldown, transient-stability, reference-stability, and tap physical-limit constraints to A_{ineq}, b_{full}.
5. **QP solve:** Call `quadprog` to solve (12). Extract:
   - $u_k = U^\star(1 : n_g)$: generator voltage increments to apply now.
   - $u^\star_{\text{OLTC},k} = U^\star(n_g{+}1 : n_g{+}n_{\text{OLTC}})$: continuous OLTC command.
   - u_{k+1} and $u^\star_{\text{OLTC},k+1}$: next-step commands (used for prediction only).
6. **Tap discretisation:** Threshold $u^\star_{\text{OLTC},k}$ to obtain the discrete command; encode in `target_v`; update cooldown counters.
7. **State prediction:** Compute $\hat{x}(k{+}1)$ and $\hat{x}(k{+}2)$ using the augmented dynamics (15) with full inputs $(u_k, u^\star_{\text{OLTC},k})$ and $(u_{k+1}, u^\star_{\text{OLTC},k+1})$ respectively.
8. **Capacitor/reactor logic:** Call `add_cap.m` with $\hat{x}(k{+}1)$ and $\hat{x}(k{+}2)$. If a switching action is selected, modify `mpc.bus(:,BS)` before the power flow.
9. **Apply generator setpoints:** Update V_{gen} in `mpc.gen`: $V_{g,i} \leftarrow V_{g,i}{+}u_k(i)$, clamped to $[V_{g,\min}, V_{g,\max}]$.
10. **Noise injection:** Add Gaussian noise to loads and generation (if enabled): $P_d \leftarrow P_{d,\text{nom}} + \mathcal{N}(0, \sigma_P^2)$, similarly for Q_d and P_g (non-slack generators). This models forecast uncertainty visible to the plant but not to the controller.
11. **Power flow:** Call `run_pf_oltc_step.m`, which runs the power flow with the updated setpoints and advances the tap by one step toward `target_v`. The function returns the new network state $x(k+1)$.

12. **State update:** Read $x(k+1)$ from the power flow results; update V_{reg} and tap-limit flags; store $u_{\text{OLTC,prev}} \leftarrow$ discretised $u^{\star}_{\text{OLTC},k}$ for the next step's augmented state.

13. **Performance logging:** Compute and log criteria via `Cal_criterion.m`.

14. **Jacobian update:** Recompute $JAC(k)$ with `makeJac` from the converged power flow results of this step. The Jacobian therefore reflects the actual post-capacitor network state. The tap increment decided at this step is *not* yet reflected: the function `run_pf_oltc_step` advances the tap *after* the power flow, so the variable `results` corresponds to the pre-advancement tap position.

7.4 Ordering Rationale

Several ordering decisions deserve attention:

- The Jacobian is recomputed at the *end* of the iteration (step 15), after the power flow, using the converged results of the current step. This ensures that $JAC(k)$ used at step $k+1$ reflects the true network operating point at the end of step k, including the effect of any capacitor switching. Computing it from the actual power flow result is more accurate than computing it before the power flow from a potentially stale state.

- The Jacobian is computed at the noisy operating point, since noise is injected before the power flow (step 10). This means the linearisation implicitly tracks the perturbed state rather than the nominal one.

- The tap advancement performed inside `run_pf_oltc_step` happens *after* the power flow; the returned `results` struct — and hence JAC — therefore correspond to the tap position before the advancement. This is what creates the two-step delay captured by the augmented state.

8 Alternative Controller Designs

8.1 OLTC Controlled by Enumeration Logic

The controller presented in the previous sections co-optimises generator voltage setpoints and OLTC tap positions within a single QP (Sections 6 and 7). An alternative design, implemented in the `Alternative OLTC logic/` folder, decouples the OLTC decision from the MPC: the tap is selected by a dedicated enumeration function `action_oltc.m`, which operates at an intermediate priority level — after the MPC but before the capacitor switching logic.

Enumeration logic. The OLTC enumeration follows the same principle as the capacitor switching logic described in Section 5. For each transformer, three candidate actions are evaluated: increase the tap ($+1$), decrease it (-1), or hold it (0). Rules can be placed on tap movements by defining eligibility conditions. A candidate action is eligible only if:

- The cooldown counter for the transformer has elapsed (no recent action on this transformer).

- The secondary bus voltage has been stable over the last $t_{\text{OLTC,last}}$ steps.
- The voltage reference for that bus will not change over the next $t_{\text{OLTC,next}}$ steps.

Among all eligible candidates, the one that minimises the criterion

$$J_{\text{OLTC}} = \|V_{\text{load,target}} - V_{\text{ref}}\|^2 + \alpha_{\text{tap}} \|Q_{\text{gen,out}}\|^2$$

evaluated at step k+2 is selected (provided it strictly improves over doing nothing and satisfies all hard constraints). As in the main controller, the physical tap limits [0.9, 1.1] are monitored via `mpc.branch(TAP)`: if the tap has reached its maximum, only a decrease is allowed; if at its minimum, only an increase.

Direction encoding. Once the enumeration selects a direction, it is encoded as a target secondary voltage passed to `run_pf_oltc_step`, using the same convention as the main controller: a target set below the current secondary voltage requests a tap increase (secondary voltage rises), and a target set above requests a tap decrease.

Two-step delay and state correction. Since the tap effect is delayed by two steps, the enumeration evaluates candidates at step k+2. The predicted state at k+2 is obtained using the matrices A and B from the state-space model (7):

$$\hat{x}(k{+}1) = A\, x_{\text{corrected}}(k) + B(k)\, u_k, \qquad \hat{x}(k{+}2) = A\, \hat{x}(k{+}1) + B(k)\, u_{k+1},$$

where $x_{\text{corrected}}(k)$ accounts for the tap that was decided at the previous step and is currently in flight:

$$x_{\text{corrected}}(k) = x(k) + \begin{pmatrix} C_{v,\text{tap}} \\ C_{q,\text{tap}} \\ 0 \end{pmatrix} \cdot step\_size \cdot direction_{k-1}.$$

This explicit correction replaces the augmented state used in the main controller. Rather than appending $u_{\text{OLTC,prev}}$ to the state vector and propagating it through $\tilde{A}$, the effect of the in-flight tap is directly added to the measured state before the QP is solved. The MPC then optimises from $x_{\text{corrected}}$ so that its generator voltage commands are consistent with the already-committed tap movement. After `action_oltc.m` selects a tap action, the predicted state $\hat{x}(k{+}2)$ is updated with the new tap's sensitivity contribution, so that the capacitor logic (priority 3) operates on a fully consistent prediction.

Modified files. The table below lists the files that differ from the main controller. All other files (`case9xx_Bsh.m`, `run_pf_oltc_step.m`, `Cal_CvCq.m`, `Cal_Cv_capCq_cap.m`, `Cal_Cv_tapCq_tap.m`, `add_cap.m`, `Cal_criterion.m`) are identical.

File	Change relative to main controller
`main.m`	Uses explicit state correction instead of augmented state. QP optimises ΔV_{gen} only (no OLTC columns). Calls `action_oltc.m` between the QP and `add_cap.m`.
`cal_mat_cst.m`	Builds simpler constant matrices without OLTC dimensions: no H_γ, no extended $\tilde{C}$; smaller H_u, H_x, b_1.
`Cal_mat_var.m`	Builds variable matrices for the reduced QP (input $u = \Delta V_{\text{gen}}$ only): A_{ineq}, H, f_1, f_2, B.
`action_oltc.m`	New file. Implements the OLTC enumeration: evaluates candidate tap actions at k+2, checks eligibility and hard constraints, selects the action minimising J_{OLTC}, encodes the direction as `target_v`, and updates $\hat{x}(k+2)$ and the cooldown counters.

Control loop. At each step k, the alternative controller executes the following sequence:

1. **State correction (tap delay):** Compute $x_{\text{corrected}}(k)$ by adding the sensitivity-predicted effect of the in-flight tap (carried over from the previous step via `oltc_idx` and `direction`) to the measured state $x(k)$.

2. **MPC — priority 1 (generator voltages):** Solve the QP from $x_{\text{corrected}}$ to obtain $u_k = \Delta V_{\text{gen}}$ and the look-ahead command u_{k+1}. Predict $\hat{x}(k+1)$ and $\hat{x}(k+2)$ using A and B.

3. **OLTC decision — priority 2:** Call `action_oltc.m` on $\hat{x}(k+2)$. If a tap action is selected, update $\hat{x}(k+2)$ with the predicted tap effect and encode the direction as `target_v`.

4. **Capacitor/reactor decision — priority 3:** Call `add_cap.m` on $\hat{x}(k+1)$ and the updated $\hat{x}(k+2)$, identically to the main controller.

5. **Actuation and power flow:** Apply u_k to generator setpoints, recompute the Jacobian, inject noise, and call `run_pf_oltc_step` with `target_v` from step 3.

The enumeration architecture has two practical advantages over the co-optimisation approach: each layer (MPC, OLTC, shunt) can be tuned and debugged independently, and the OLTC logic remains operational even when `quadprog` fails, since it operates entirely outside the QP. Its main limitation is that the generator voltage commands are computed without knowledge of the upcoming tap action, so the two decisions are optimised sequentially rather than jointly. Whether this suboptimality is significant depends on the operating condition; in the simulations of Section 9 the performance gap is small when $\beta > 0$ (see Section 9.2).

8.2 MIQP with Integrated Capacitor Switching

Formulation. The enumeration logic described in Section 5 operates *after* the MPC and selects a switching action greedily, without accounting for the interaction between the capacitor action and the optimal generator voltage setpoint. Integrating capacitor

switching directly into the MPC would allow the two actuators to be co-optimised simultaneously.

The capacitor switching action at bus i is modelled as an integer control input $u_{\text{cap},i}(k) \in \{-1, 0, +1\}$, where $+1$ (resp. -1) corresponds to adding one capacitor (resp. one reactor) of susceptance `cap`. The extended input vector becomes:

$$u_{\text{ext}}(k) = \begin{pmatrix} \Delta V_{\text{gen}}(k) \\ u_{\text{cap}}(k) \end{pmatrix} \in \mathbb{R}^{n_g} \times \{-1, 0, +1\}^{n_t}.$$

The state-space model $x(k+1) = A\,x(k) + B_{\text{ext}}(k)\,u_{\text{ext}}(k)$ retains the same $A = I$ and output matrix C as (7); only the input matrix is extended:

$$B_{\text{ext}}(k) = \begin{pmatrix} C_v(k) & \texttt{cap} \cdot C_{v,\text{cap}}(k) \\ C_q(k) & \texttt{cap} \cdot C_{q,\text{cap}}(k) \\ I_{n_g} & 0 \end{pmatrix}. \tag{24}$$

The prediction matrices Π and Γ generalise straightforwardly by replacing B with B_{ext}.

Advantages. Co-optimising the capacitor decision with the generator voltage setpoints within a single problem offers two advantages over the greedy enumeration:

- **Simultaneous coordination:** The MPC accounts for the effect of a capacitor action when computing the generator voltage commands. In situations where a switching action significantly shifts the reactive power distribution, the resulting generator setpoints are better adapted than those produced by a sequential approach.
- **Unified constraint handling:** Rather than checking hard constraints after the fact, the MPC enforces them within the optimisation. Soft constraints on reactive power margins ($|Q_{\text{gen}}| \leq Q_{\text{lim}}$) can be added to the cost function with an appropriate weight α_{lim}, providing a unified treatment of voltage tracking and margin preservation. Enforcing them as hard constraints would risk infeasibility in stressed operating conditions; as a soft penalty they bias the solution toward margin-preserving decisions without that risk.

Why capacitors cannot be discretised like OLTC taps. The quasi-continuous approximation used for OLTC taps (Section 7.1) relies on the step size being small relative to the total range: with 33 levels over [0.9, 1.1], thresholding a continuous QP variable at $\pm\varepsilon$ introduces negligible suboptimality. Capacitor switching does not satisfy this condition: the action space is $\{-1, 0, +1\}$ which corresponds to $\{-cap, 0, +cap\}$, a coarse three-point set with no intermediate levels. Treating u_{cap} as continuous and rounding the result would introduce large and unpredictable suboptimality — the continuous optimum might lie at 0.4, which rounds to 0 (no action), while $+1$ could have been the globally optimal discrete choice. Adding 0.4 capacitor could be enough to ensure sufficient margin. In this situation a controller with MIQP won't do anything, but an operator would have. The quasi-continuous approximation is therefore not applicable, and the binary nature of the decision must be handled explicitly.

Why MIQP for tap changers is not interesting. Even setting aside the unpredictable solve times of branch-and-bound, using MIQP for OLTC taps would not be beneficial. To keep the computational cost bounded, integer variables would need to be restricted to the first step of the prediction horizon (i.e., $u_{\text{OLTC}}(k+j) = 0$ for $j \geq 1$). But the primary benefit of integrating the OLTC into MPC is precisely the multi-step lookahead: the optimiser plans a sequence of tap movements in advance, anticipating the two-step delay of each action. Restricting integer decisions to a single horizon step eliminates this lookahead and reduces the OLTC decision to a greedy single-step choice — equivalent in information content to the enumeration logic. At that point, the quasi-continuous approach (33 discrete levels, negligible rounding error, standard convex QP) is strictly preferable: it retains the full N-step lookahead at no integer programming cost.

Computational tractability and its limits. With $n_t = 3$ load buses and a horizon of N steps, a fully unconstrained MIQP over the capacitor variables would involve $3^{n_t \cdot N}$ integer combinations — $3^9 = 19\,683$ for $N = 3$. To keep the problem tractable, two restrictions are natural: allow capacitor switching only on the first step of the prediction horizon ($u_{\text{cap}}(k+j) = 0$ for $j \geq 1$), and allow at most one switching action at step k. These restrictions reduce the integer search to $2n_t + 1 = 7$ candidates — exactly the same cardinality as the greedy enumeration. The branch-and-bound tree degenerates to a single level, making the MIQP roughly equivalent in computational cost to the current approach.

This similarity also reveals the main limitation: the multi-step planning benefit — the ability to schedule sequences of capacitor actions over the horizon to jointly optimise voltage tracking and reactive margins — is entirely lost when switching is confined to the first step. What remains is the co-optimisation benefit (simultaneous solution with generator voltages), which is real but modest: in most operating conditions, the enumeration already selects the same action that the MIQP would choose. The practical gain over the current architecture is therefore limited.

Required dynamic constraints. Embedding capacitor switching in the MPC requires replicating, as hard constraints in the MIQP, the eligibility conditions currently handled procedurally in `add_cap.m`:

- **Cooldown:** If the cooldown counter for bus i is active at step k, force $u_{\text{cap},i}(k) = 0$.
- **Past voltage stability:** If the load voltage at bus i has been unstable over the last $t_{\text{cap,last}}$ steps, block switching at that bus.
- **Future reference stability:** If the voltage reference will change within the next $t_{\text{cap,next}}$ steps, block switching at that bus.

These conditions are evaluated before the MIQP solve; buses that fail them are fixed to $u_{\text{cap},i}(k) = 0$, reducing the integer search space further.

A parallel fallback architecture. Because a MIQP solver may time out or fail to find a feasible integer solution within the control sampling period, the MIQP controller cannot be deployed as the sole controller. A practical architecture could run it in parallel with the standard QP-based controller:

- At each step k, both controllers solve their respective problems simultaneously, with the MIQP given a strict time budget.
- If the MIQP returns a feasible solution within the budget, its result is applied: generator setpoints, tap commands, and capacitor actions are all taken from the MIQP output.
- If the MIQP times out, fails to find a feasible solution, or returns a result that violates a hard constraint, the fallback QP controller's output is applied instead.

This architecture guarantees that a valid control action is always applied within the sampling period. The fallback QP is not a degraded mode but a continuously warm-started backup running at every step regardless of the MIQP outcome.

9 Simulation Results

9.1 Performance Criterion

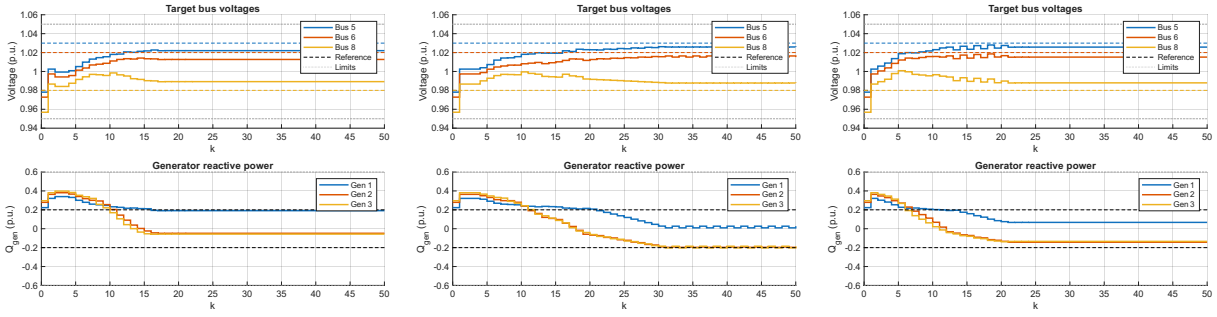
All simulations are evaluated using the voltage tracking criterion

$$J_V = \sum_{i \in \mathcal{I}_{\text{target}}} \left(V_{\text{load},i}(k) - V_{\text{ref},i} \right)^2, \tag{25}$$

where $\mathcal{I}_{\text{target}}$ is the set of load buses with active consumption (buses 5, 6, and 8), and $V_{\text{ref},i}$ is the voltage reference assigned to bus i.

9.2 Controller Comparison

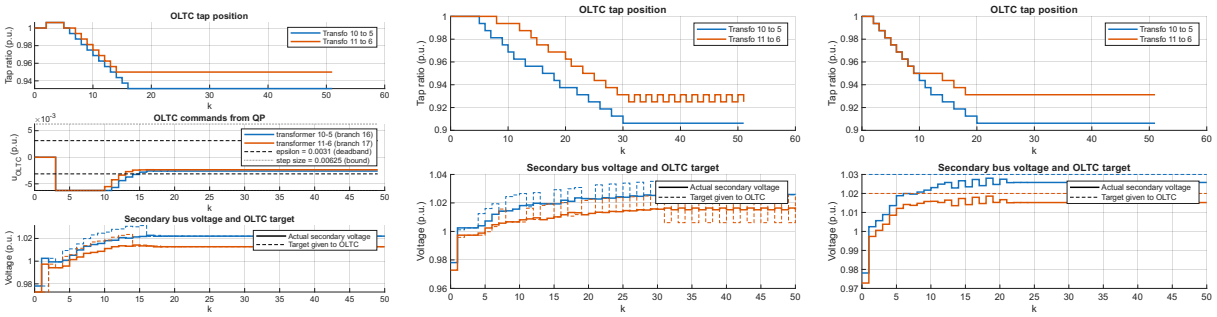
Penalised tap configuration ($\beta = 1$). Figures 3 and 4 compare the voltage tracking, generator reactive power, and OLTC tap behaviour across the three controllers for $\beta = 1$ (tap penalization term). Figure 5 summarises the resulting cumulative criterion J_V.



(a) Main controller (b) Alternative OLTC logic (c) Naive controller

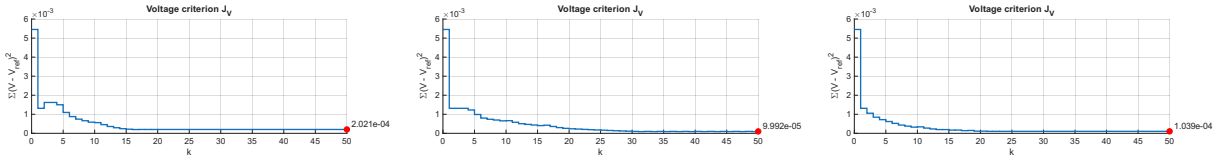
Figure 3: Load bus voltages (top) and generator reactive powers (bottom) for the three controllers ($\beta = 1$). All buses converge to values close to their references (dashed lines). Bus 8 shows a small persistent offset across all controllers.

All three controllers successfully regulate the load bus voltages towards their respective references (Figure 3). Buses 5 and 6 converge closely to their targets; bus 8 shows a small persistent offset. This offset is structural: the two OLTC transformers act on buses 5 and 6 (branches 10→5 and 11→6), which are geographically distant from bus 8 in the



(a) Main controller (b) Alternative OLTC logic (c) Naive controller

Figure 4: OLTC tap position and secondary bus voltage for the three controllers ($\beta = 1$). The main controller stops the tap earlier (at ≈0.93–0.95) due to the $\beta\|u_{\text{OLTC}}\|^2$ penalty, while the alternative and naive controllers drive the tap further down (toward 0.9–0.925), improving secondary voltage tracking.



(a) Main ($J_V = 2.02 \times 10^{-4}$) (b) Alternative ($J_V = 9.99 \times 10^{-5}$) (c) Naive ($J_V = 1.04 \times 10^{-4}$)

Figure 5: Cumulative voltage criterion J_V for the three controllers ($\beta = 1$). All three converge to values of the same order of magnitude. The main controller's final value is approximately twice that of the alternative and naive controllers, a direct consequence of the tap penalty in its cost function.

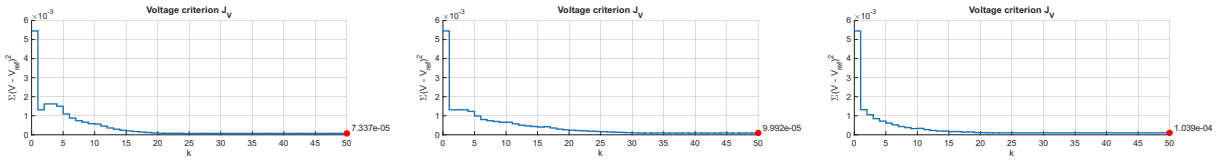
63 kV network. Bus 8 is located at the centre of the sub-transmission ring and receives tap action only indirectly, through the propagation of voltage changes along the lines 7–8 and 8–9. The generator voltage commands can partially compensate, but the MPC cost function weights buses 5, 6, and 8 equally, so the solver preferentially exploits the directly controlled buses and accepts a residual error on bus 8. The criteria converge to similar orders of magnitude across all three implementations, confirming that all three architectures are effective at voltage regulation.

Their respective design advantages are the following. The **main controller** co-optimises generator voltages and OLTC tap increments in a single QP, yielding a globally optimal solution at each step. The two-step tap delay is modelled directly through the augmented state $\tilde{x}$ and transient tap protection is enforced as hard QP constraints, providing a unified framework where the performance/actuation trade-off is encoded in the cost weight β. The **alternative OLTC logic** decouples the tap decision into a dedicated enumeration layer (`action_oltc.m`), completely separate from the MPC. This makes each layer easier to tune and debug independently; the two-step delay is compensated by an explicit state correction before the QP solve. Tap oscillations visible in Figure 4 (centre) reflect the enumeration logic chasing a slowly evolving target, but the overall tracking performance is good. The **naive controller** is the simplest implementation: the tap advances one step whenever the secondary bus voltage deviates from V_{ref}, with no predictive anticipation of the two-step delay. The resulting tap trajectory is smooth and monotone, and this controller serves as a natural baseline to quantify the benefit of more sophisticated strategies.

The main controller's final criterion ($J_V = 2.02 \times 10^{-4}$) is approximately twice that of the alternative (9.99×10^{-5}) and naive (1.04×10^{-4}) controllers. This is a direct consequence of the $\beta\|u_{\text{OLTC}}\|^2$ term with $\beta = 1$: the QP deliberately limits tap usage to keep the penalised cost low, stopping the tap at $\approx$0.93–0.95 around step $k \approx 15$–18, whereas the alternative and naive controllers continue to drive the tap further down toward 0.9, providing additional performance in voltage tracking. A secondary effect also visible in Figure 4 is a *hesitation* phenomenon: the quadratic penalty induces the MPC to spread the tap command over the prediction horizon rather than concentrating it at the current step. The resulting per-step command $u_{\text{OLTC},k}$ is systematically just below the discretisation threshold $\varepsilon = \text{step\_size}/2$, so no tap step is triggered even when a single increment would be beneficial. This is a direct consequence of the quadratic nature of the penalty: unlike an ℓ_1 norm, which would concentrate the effort into a single large step above the threshold, the ℓ_2 norm penalises large commands disproportionately and naturally spreads the effort into many small sub-threshold increments.

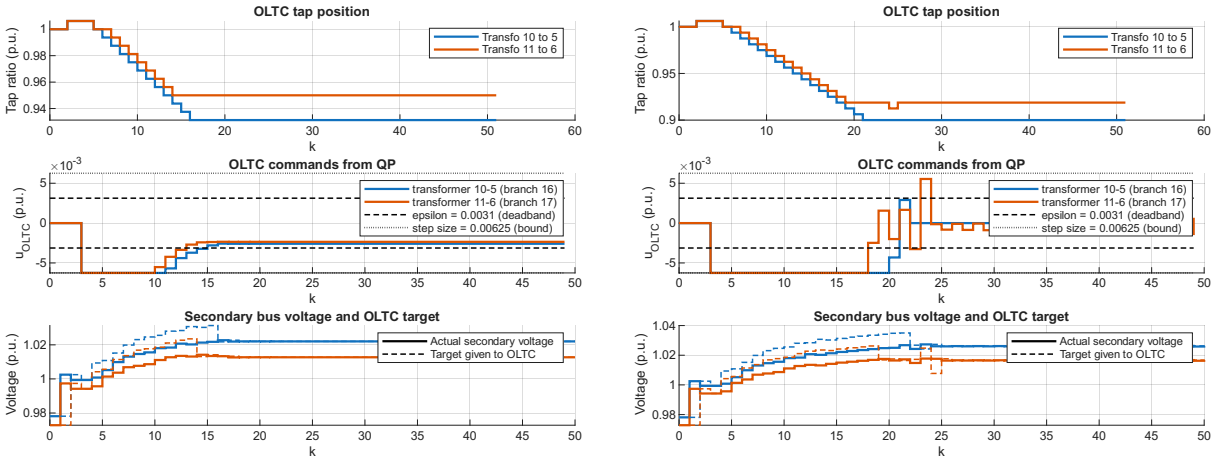
Effect of removing the tap penalty ($\beta = 0$). Setting $\beta = 0$ removes the tap penalty from the main controller's cost function, allowing the QP to freely co-optimise tap increments to minimise the voltage tracking error only. Figure 6 compares the resulting criteria for all three controllers, and Figure 7 contrasts the main controller's tap behaviour between $\beta = 1$ and $\beta = 0$.

With $\beta = 0$, the main controller achieves $J_V = 7.34 \times 10^{-5}$, clearly outperforming both the alternative (9.99×10^{-5}) and naive (1.04×10^{-4}) controllers. The criteria of the alternative and naive controllers are identical between $\beta = 0$ and $\beta = 1$ (as expected, since β does not appear in their cost functions), which confirms that co-optimising generators and taps in a single QP is strictly superior to any decoupled approach when no tap penalty is applied.



(a) Main ($J_V = 7.34 \times 10^{-5}$) (b) Alternative ($J_V = 9.99 \times 10^{-5}$) (c) Naive ($J_V = 1.04 \times 10^{-4}$)

Figure 6: Cumulative voltage criterion J_V for the three controllers with $\beta = 0$. The main controller now achieves the best criterion (7.34×10^{-5}), outperforming both the alternative and naive controllers. The criteria of the alternative and naive controllers are unchanged relative to $\beta = 1$, confirming that the performance gap observed earlier was entirely due to the cost weight, not to an algorithmic limitation.



(a) Main controller, $\beta = 1$ (b) Main controller, $\beta = 0$

Figure 7: Tap behaviour of the main controller with ($\beta = 1$, left) and without ($\beta = 0$, right) tap penalisation. Without the penalty, the QP drives the tap to the lower bound of 0.9 and the commands u_{OLTC} become more variable around $k \approx 18$–25 as the optimiser exploits the tap more aggressively.

However, this performance gain comes at a cost: as visible in Figure 7, the OLTC commands from the QP become more variable at $\beta = 0$, exhibiting oscillatory behaviour around $k \approx 18$–25. In practice, each tap operation represents a physical mechanical switching action on the transformer. OLTCs have a finite lifespan measured in the number of switching operations, and unnecessary oscillations accelerate wear and increase maintenance costs. Penalising the tap magnitude with $\beta > 0$ naturally suppresses such chattering.

There is therefore a fundamental trade-off between voltage tracking performance and actuator economy. The weight β is the direct tuning knob for this trade-off: $\beta = 0$ maximises tracking performance at the expense of tap activity, while large β sacrifices some tracking performance to reduce tap usage. Finding an appropriate value — along with the other cost weight α — requires a multi-objective calibration that accounts for the operator's priorities and the transformer's maintenance budget.

Table 1 summarises the final cumulative criterion J_V for all three controllers under both penalty settings.

Table 1: Final cumulative voltage criterion J_V for the three controllers.

	Main controller	Alternative OLTC	Naive controller
$\beta = 1$	2.02×10^{-4}	9.99×10^{-5}	1.04×10^{-4}
$\beta = 0$	7.34×10^{-5}	9.99×10^{-5}	1.04×10^{-4}

The alternative and naive controllers are unaffected by β since this weight does not appear in their cost functions. With $\beta = 0$ the main controller achieves the best criterion, confirming the benefit of co-optimisation; with $\beta = 1$ it trades tracking performance for reduced tap usage.

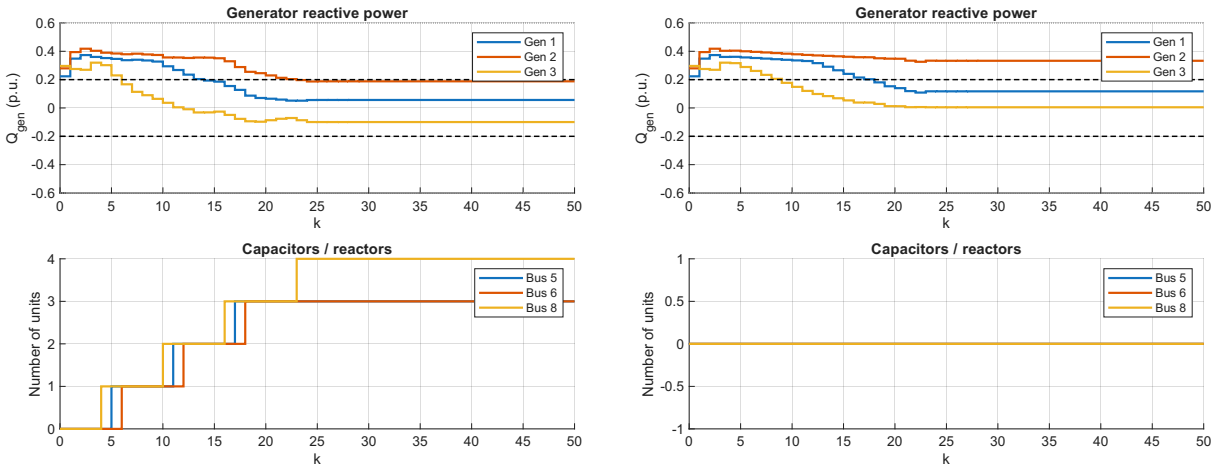
9.3 Role of Shunt Capacitors and Reactors

Reactive margin recovery. The shunt capacitor/reactor logic (`add_cap.m`) operates as a lower-priority layer whose sole purpose is to bring the generator reactive powers back within the soft limit $[-Q_{\text{lim}}, Q_{\text{lim}}]$. Figure 8 compares the generator reactive powers and shunt switching activity between a simulation with and without this layer enabled.

Without shunt switching, the MPC and OLTC regulate the load bus voltages but cannot simultaneously bring all generator reactive powers within $[-Q_{\text{lim}}, Q_{\text{lim}}]$: Gen 2 in particular remains stuck at $\approx$0.33 p.u., well above the soft limit of 0.2 p.u., for the entire simulation horizon. This is a structural limitation: the generator voltages and the tap are the only control levers available to the MPC and OLTC, and the network reactive demand forces them to operate with reduced margins.

With shunt switching enabled, capacitors are added progressively at buses 6 and 8 (up to 3 and 4 units respectively by $k \approx 25$), injecting local reactive power directly at the load buses. This offloads the reactive burden from the generators, which converge toward the $[-Q_{\text{lim}}, Q_{\text{lim}}]$ window. Preserving these margins is essential to the project's primary safety objective: a generator operating close to its hard reactive limit $Q_{\max}$ or $Q_{\min}$ has little headroom to absorb a sudden disturbance, increasing the risk of automatic disconnection.

Effect on voltage tracking. Figure 9 shows the load bus voltages for the same two cases. The voltage tracking trajectories are nearly identical: buses 5, 6, and 8 converge



(a) With shunt switching (`add_cap = true`)

(b) Without shunt switching (`add_cap = false`)

Figure 8: Generator reactive powers (top) and shunt unit count per bus (bottom), with and without the capacitor/reactor layer. Dashed lines mark the soft limit $\pm Q_{\text{lim}} = \pm 0.2$ p.u. Without shunt switching, Gen 2 remains persistently above Q_{lim} throughout the simulation. With shunt switching, capacitors are progressively added at buses 6 and 8, relieving the generators and allowing them to return within the soft limits.

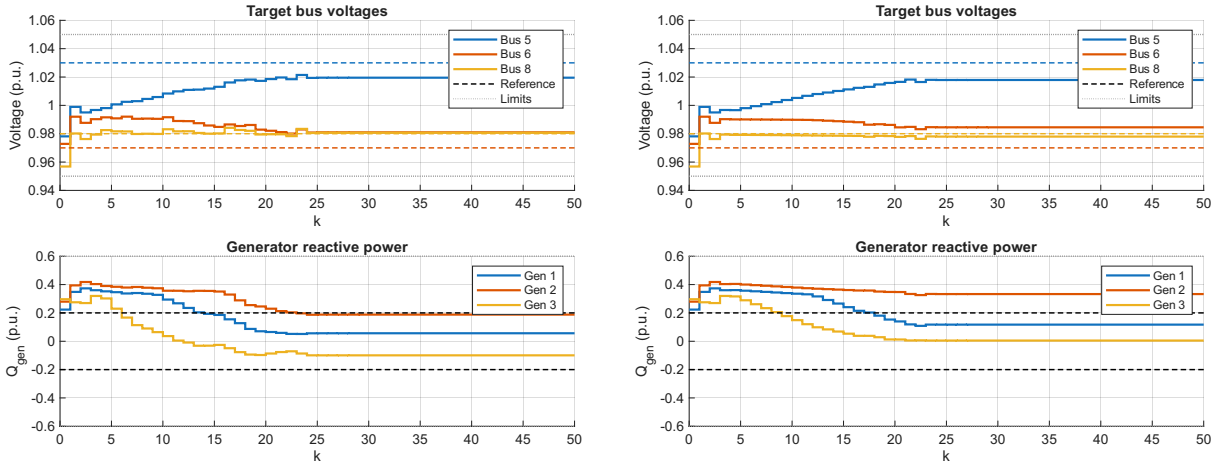
to the same steady-state values regardless of whether shunt switching is active. This confirms that the capacitor layer does not interfere with the primary voltage regulation objective — it adds value exclusively on the reactive margin dimension, orthogonal to the tracking performance already achieved by the MPC and OLTC.

9.4 Effect of Noise on OLTC Behaviour

Noise-induced tap oscillations. The simulations presented so far used zero process noise. Figure 10 and Figure 11 show the behaviour of the main controller under Gaussian white noise of amplitude 5% of the nominal load consumption and renewable generation at each step.

The oscillation mechanism is the following. The MPC produces a command $u_{\text{OLTC},k} \approx 0$ for transformer 11→6, meaning it considers the current tap position adequate. However, at each step the function `run_pf_oltc_step` advances the tap by one step whenever the measured secondary bus voltage deviates from the target given to the OLTC by more than the tolerance `tol`. When noise shifts the secondary voltage across this threshold — independently of the MPC decision — a tap step is triggered without the controller's knowledge. This step modifies the actual network state, which at the next step again differs from the (noise-free) target, potentially triggering another step in the opposite direction. The result is the back-and-forth tap movement visible in Figure 10: the MPC sees a noisy state, computes a near-zero command, but the reactive noise on loads and generation continuously perturbs the secondary bus voltage across the tolerance boundary.

This interaction reveals a structural weakness of the architecture: the tap advancement logic inside `run_pf_oltc_step` operates on the *measured* secondary voltage, which is corrupted by noise, while the MPC plans on a noise-free prediction. There is no coordination between these two mechanisms, so noise can drive the tap in a direction that the MPC would not have chosen. Increasing `tol` would reduce the sensitivity to noise at



(a) With shunt switching (`add_cap = true`)

(b) Without shunt switching (`add_cap = false`)

Figure 9: Load bus voltages (top) and generator reactive powers (bottom), with and without the capacitor/reactor layer. The voltage tracking trajectories are nearly identical between the two cases, confirming that the shunt layer does not interfere with the MPC voltage regulation objective.

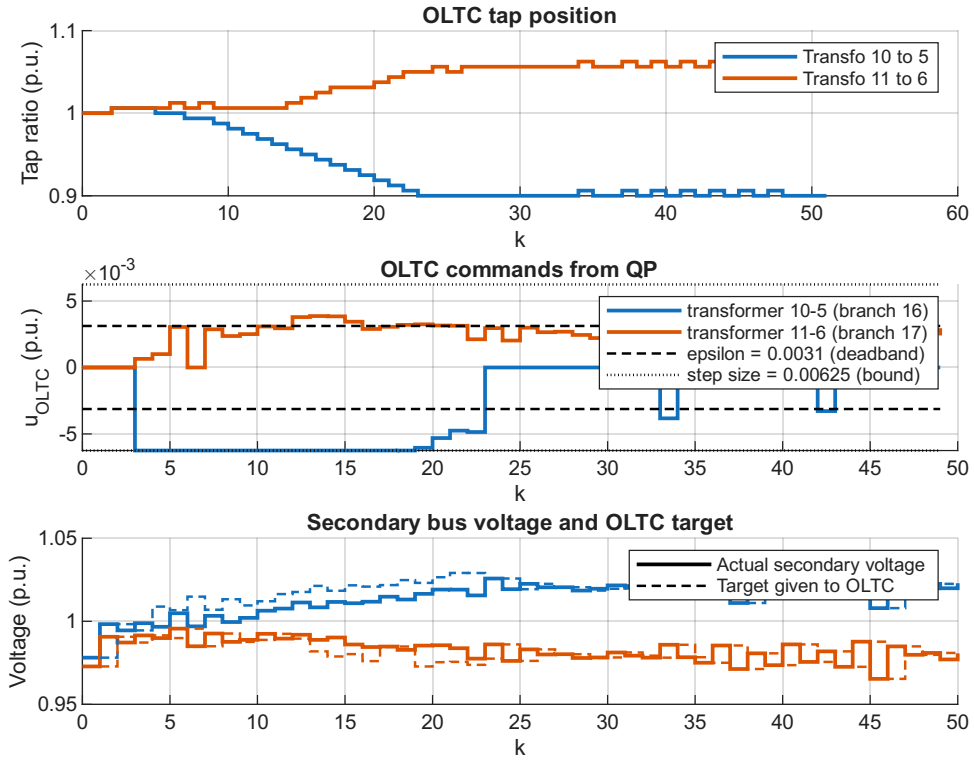


Figure 10: OLTC tap position, QP commands, and secondary bus voltage under 5% noise. Transformer 10→5 (blue) descends monotonically to the lower bound as expected. Transformer 11→6 (orange) first rises to ≈1.06 then oscillates back and forth after $k \approx 30$, while the corresponding QP command u_{OLTC} remains near zero — the MPC has decided no tap movement is needed, yet the tap keeps changing.

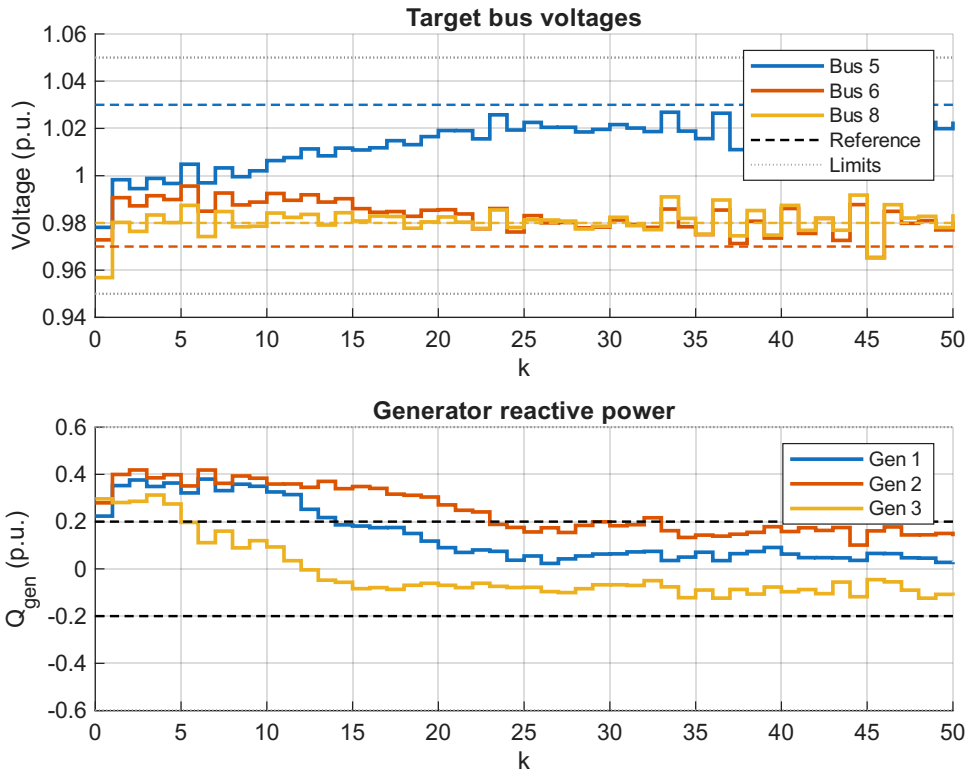


Figure 11: Load bus voltages and generator reactive powers under 5% noise. All buses converge to near-reference values despite the noise, but exhibit persistent step-like fluctuations caused by the noise-driven tap oscillations.

the cost of looser secondary voltage regulation; a more principled solution would be to filter the secondary voltage measurement or to incorporate noise explicitly into the MPC formulation (stochastic or robust MPC).

10 Critical Analysis

Linearisation and model validity. The controller relies on a local linearisation of the power flow equations around the current operating point, recomputed at every step from the Jacobian $JAC(k)$, which is not realistic. When sensitivity matrices are not computed often enough, the use of the controller becomes counter productive. Indeed this sensitivity model is only accurate for small deviations from the operating point; large disturbances or change in the grid topology (e.g. shunt addition) push the system into regions where the linear prediction deteriorates. Furthermore, the assumption $A = I$ — that voltages and reactive powers remain constant without control action — ignores the dynamics of the primary AVR loop, which in reality produces transients on the time scale of secondary control. The model also assumes that each generator tracks its voltage setpoint perfectly and instantaneously, whereas AVRs have their own dynamics.

Robustness and constraint satisfaction. No formal closed-loop stability analysis is provided, and recursive feasibility of the QP is not guaranteed: if the constraints become mutually incompatible (for instance, $Q_{\max}$ too tight relative to the voltage bounds over a short horizon), the solver returns infeasible and no control action is issued. The current implementation has no fallback mechanism in this case. Furthermore, the Gaussian noise

injected at each step on loads and generation is not modelled by the controller; in an adverse realisation, the noise shifts the true state outside the region predicted by the QP, potentially causing the actual trajectory to violate hard constraints ($V_{l,\min}/V_{l,\max}$, $Q_{\min}/Q_{\max}$) even when the optimised predicted trajectory satisfies them. The alternative controller architecture described in Section 8 partially addresses the QP failure issue: the enumeration-based OLTC logic and the QP-only fallback together ensure that a valid control action is always available.

Simulation scenario. The simulation uses Gaussian white noise on load consumption and renewable generation, which is a simplified disturbance model: real perturbations (wind variations, cloud cover) are temporally and spatially correlated. More importantly, the nominal active and reactive consumption at each load bus, and the active production of each generator, are kept constant throughout the simulation. Real operating conditions involve slow ramps, daily demand cycles, and generation dispatch changes, which would test the controller's ability to track a moving equilibrium rather than a fixed operating point. The simulation also does not test any network contingency: no line outage and no generator disconnection are simulated. These events would abruptly change the power flow topology, invalidating the current sensitivity matrices and potentially driving the system into a regime far outside the linearisation validity, which constitutes the most critical stress test for a secondary voltage controller.

OLTC interface and MPC hesitation. In the main controller, the OLTC command u_{OLTC} is penalised by the term $\beta\|u_{\text{OLTC}}\|^2$ in the cost function to limit tap movement. This quadratic penalty may cause the MPC to redistribute the tap increase across the entire prediction horizon leading to hesitation. Moreover the criterion does not penalise tap oscillation which accelerates the aging of OLTC. Additionally, the current implementation reads `mpc.branch(TAP)` to detect whether a tap has reached its physical limits [0.9, 1.1] and adds a unilateral constraint accordingly. In a real substation, this information is typically not available to the secondary controller: the tap position is not directly observable. Finally, a real OLTC control interface does not accept an arbitrary target voltage: it accepts one of three discrete secondary voltage targets ($V_{\text{target}} \in \{0.95, 1.00, 1.05\}$ p.u.), corresponding to lower, hold, and raise orders respectively. The current implementation assumes a richer interface, which overstates the controller's practical authority over the tap. The alternative OLTC enumeration controller (Section 8.1) avoids the hesitation issue by design: the tap is moved only when it strictly reduces the criterion, with no magnitude penalty.

Suboptimality of the capacitor logic. The enumeration logic in `add_cap.m` applies at most one switching action per step, selected greedily after the MPC has already computed the generator setpoints. As a result, the generator voltage commands do not account for the upcoming capacitor action, and the capacitor action does not account for the interaction with future generator commands. This sequential, single-action approach is provably suboptimal relative to a joint optimisation. Section 8.2 discusses how integrating capacitor switching into a MIQP formulation would address this limitation, at the cost of significantly increased computational complexity. Additionally, only one capacitor or reactor can be switched per time step, and the enumeration logic operates with no predictive horizon — it reacts greedily to the current predicted state rather than

anticipating future reactive power needs. This can lead to a slower restoration of reactive margins, particularly when several buses simultaneously require reactive support.

11 Perspectives

The following directions would either address limitations identified in the critical analysis or extend the controller towards a more complete and industrially deployable system.

11.1 Improvements to the Current Architecture

Online Jacobian estimation. The current controller recomputes the Jacobian analytically at each step from the converged power flow solution. In practice, a perfect power flow solution may not always be available in real time. An alternative would be to maintain a state-space representation of the network and update the Jacobian online through a dedicated state observer, using only the measurements. This would make the linearisation more robust to measurement gaps and reduce the controller's dependence on a centralised power flow solver.

Tap position observer. The current implementation reads `mpc.branch(TAP)` to detect whether a tap has reached its physical limits — information that is not directly observable in practice. A dedicated tap position observer, updated from secondary voltage measurements combined with the known tap step size and issued direction commands, would remove this unrealistic assumption and is a prerequisite for any real deployment.

Tap penalty. The quadratic penalty $\beta\|u_{\text{OLTC}}\|^2$ in the cost function encourages the MPC to distribute the total tap displacement evenly across the prediction horizon, producing small fractional commands at every step rather than committing to a discrete action at the earliest opportunity. Replacing this term with a linear (L1) penalty $\beta\|u_{\text{OLTC}}\|_1$ promotes sparsity: the optimiser prefers to concentrate the tap action into as few steps as possible, which is physically more appropriate for a discrete mechanical actuator and avoids the hesitation phenomenon described in Section 10. Furthermore, if the MPC continues to produce sub-threshold tap commands despite this modification, the enumeration-based OLTC logic (as implemented in the alternative controller of Section 8.1) could be used as a tie-breaker to make the final decision. Moreover the controller may generate tap oscillations, penalizing tap changes could fix the problem by adding a term $\|u_{\text{OLTC}}(k) - u_{\text{OLTC}}(k-1)\|^2$ to the cost function, discouraging rapid direction reversals. Another possibility would be to introduce a deadband on the voltage tracking error below which the tap is not modified.

Multiple simultaneous capacitor switches. The enumeration logic currently applies at most one switching action per step to limit actuator wear and control chattering. However, when the number of buses equipped with switchable shunt elements is small — as in the simulated 12-bus network, where only three buses carry shunt elements — restricting to a single action per step is a severe limitation: if all three buses simultaneously require reactive support, the controller needs at least three steps to respond. In such cases, allowing two or more simultaneous switches, still selected by enumeration but jointly evaluated, could significantly accelerate the restoration of reactive margins. The combinatorial enumeration grows as $O(2^{n_{\text{cap}}})$, which remains tractable for small n_{cap}.

QP fallback using the enumeration architecture. Because the OLTC enumeration of the alternative controller operates entirely outside the QP, it remains functional even when `quadprog` fails (e.g. due to conflicting constraints in an extreme operating condition or numerical conditioning issues). A practical fallback strategy for the main controller would be: upon QP failure, re-solve a reduced QP over ΔV_{gen} only (identical to the alternative controller's QP, with the OLTC columns removed). This smaller problem has fewer variables and fewer OLTC-related constraints, making it strictly less likely to be infeasible. The OLTC decision is then recovered by calling the enumeration logic on the predicted state. This two-tier fallback guarantees that, even in the worst case, the controller degrades gracefully to the alternative architecture rather than applying zero input.

Parallel controller architecture. To balance optimality against computational reliability, a parallel architecture can be considered. Three controllers run independently at each step:

- **MIQP controller:** integrates capacitor switching as binary variables for global optimality. Its solution is applied only if the solver returns within the sampling period; otherwise it is discarded.
- **Current QP controller:** the main controller described in this report, used by default.
- **No-OLTC fallback:** a simplified QP that optimises generator voltages only, with no tap variables. Its solution is applied whenever the main QP returns infeasible, guaranteeing that a valid control action is always issued.

This architecture provides graceful degradation: the system is always controlled, with the best solution available within the computational budget.

11.2 Extensions of the Control Framework

Hierarchical coupling with a tertiary controller. The secondary controller described in this report tracks fixed voltage references V_{ref}. In a full hierarchical architecture, these references would be provided by a tertiary level solving an Optimal Power Flow (OPF) on a slower time scale (minutes to hours), optimising an economic objective such as active power losses or generation cost. Connecting the two levels would allow the secondary controller to pursue voltage tracking objectives that are globally consistent with the network optimum.

Stochastic MPC with renewable forecasts. The current disturbance model uses Gaussian white noise. Real renewable perturbations (wind gusts, cloud cover) are temporally correlated and can be partially anticipated using short-term probabilistic forecasts. A scenario-based stochastic MPC would propagate an ensemble of forecast trajectories through the prediction model and compute a control policy that minimises expected cost while satisfying constraints with a given probability, providing formal robustness guarantees against renewable variability.

Distributed multi-zone coordination. The current controller is centralised and operates on a single 12-bus zone. A realistic sub-transmission network comprises multiple geographically separated zones with electrical coupling at boundary buses. Extending the architecture to a distributed MPC framework would allow each zone to solve its local QP independently while exchanging boundary voltage and reactive power information with neighbouring zones, scaling the approach to large networks without requiring a centralised solver.

Coupling with an active power controller. Voltage control and active power dispatch are traditionally treated as separate problems, but in networks with high renewable penetration they are tightly coupled: changes in active power injection (curtailment, ramp-rate limiting) significantly affect reactive power flows and voltage profiles. Coupling the secondary voltage controller with an active power controller through a hierarchical cascade would allow reactive and active degrees of freedom to be exploited jointly, improving both voltage regulation and overall system efficiency.

12 Conclusion

This report has presented the design of a secondary voltage controller for a subtransmission network integrating renewable generation, developed in partnership with RTE. The controller combines three complementary layers operating at distinct time scales and priorities: an MPC that issues generator voltage setpoints via a receding-horizon QP, an OLTC action that adjusts transformer tap ratios, and a shunt capacitor/reactor switching logic that provides coarse reactive power support.

A central challenge in integrating on-load tap changers into an optimisation-based controller is the nonlinear nature of the relationship between an OLTC's target secondary voltage and the resulting network state. The power flow equations are nonlinear, and the tap ratio enters them multiplicatively: specifying a target voltage does not define a linear mapping to load voltages or reactive powers. Directly including the target voltage as an optimisation variable would therefore break the convex QP structure.

This difficulty is circumvented by shifting the decision variable from the tap target to the tap *increment* $u_{\mathrm{OLTC}} \in [-step\_size, step\_size]$. Because the step size is small relative to the operating range ($step\_size = 0.00625\,\mathrm{p.u.}$ over a range of $[0.9, 1.1]$), the effect of one tap increment on load voltages and reactive powers is well captured by the linear sensitivity matrix $C_{v,\mathrm{tap}}$ derived from the Jacobian. The nonlinear actuator is thus replaced by a locally linear model: $\Delta V_{\mathrm{load}} \approx C_{v,\mathrm{tap}}\, u_{\mathrm{OLTC}}$. This approximation is renewed at every step through the Jacobian update, maintaining accuracy despite changing operating conditions.

Within this linearised framework, the main controller co-optimises generator voltage increments ΔV_{gen} and tap increments u_{OLTC} in a single QP. The two-step actuation delay of the tap is captured by an augmented state that propagates the in-flight command through the prediction model, enabling the MPC to issue anticipatory commands rather than reacting after the effect has appeared. A quadratic penalty $\beta\|u_{\mathrm{OLTC}}\|^2$ in the cost function ensures that the tap is only engaged when the generator voltages alone are insufficient, achieving a global co-optimisation across all degrees of freedom.

Many directions remain open. On the modelling side, the sensitivity matrices are currently computed analytically from a converged power flow, which may not be avail-

able in real time; identifying them online from measurements would bring the architecture closer to a deployable system. Increasing the realism of the simulation scenario is equally important: constant nominal operating points, uncorrelated Gaussian noise, and the absence of network contingencies are significant simplifications relative to actual sub-transmission conditions. Testing the controller against slow demand ramps, generation dispatch changes, and line outages would provide a more faithful assessment of its robustness and of the limits of the local linearisation.

A Simulation Parameters

Parameter	Symbol	Value
Load voltage reference	V_{ref}	Configurable
Prediction horizon	N	3 steps
Max generator voltage increment	$u_{\max}$	0.2 p.u./step
Voltage tracking weight	—	1 (implicit)
Reactive power weight	α	0.001
OLTC magnitude weight	β	1
Tap step size	*step_size*	0.00625 p.u.
Tap range	$[a_{\min}, a_{\max}]$	[0.9, 1.1]
Tap deadband	`tol`	0.00625/2
OLTC discretisation threshold	ε	*step_size*/2
Soft reactive limit	Q_{lim}	0.2 p.u.
Capacitor size	`cap`	5 Mvar
Capacitor cooldown	t_{cap}	5 steps
Past voltage stability window	$t_{\text{cap,last}}$	3 steps
Past voltage stability threshold	`var_v_cap_last`	0.01 p.u.
Future reference stability window	$t_{\text{cap,next}}$	3 steps
Future reference stability threshold	`var_v_cap_next`	0.01 p.u.
Jacobian recalculation period	`t_jac`	0 (every step)
Noise rate	σ_{rate}	0.05 (5%)

The voltage reference V_{ref} is a design parameter that can be set independently per load bus; all three buses use the same value in the default simulation scenario.

B File Organisation

The project is organised into four folders. Files shared across controller folders have identical content.

`Main controller/`

`main.m` Main script. Defines all parameters, initialises the network, runs the simulation loop, and produces plots. Generator voltages and OLTC taps are co-optimised in a single QP using an augmented state. The Jacobian is recomputed with `makeJac` at the *end* of each iteration, from the converged power flow results of that step; the recalculation period is controlled by `t_jac` (0 = every step, n = every n steps).

cal_mat_cst.m Builds MPC matrices that do not depend on the operating point: C, Ψ, $\tilde{C}$, H_u, H_x, b_1, H_γ (including OLTC dimensions). Called once at initialisation.

Cal_mat_var.m Builds MPC matrices that depend on the operating point: $\tilde{A}$, $\tilde{B}$, Π, Γ, H, f_1, f_2, A_{ineq}, b_2. Called at every step.

Alternative OLTC logic/

main.m Main script for the alternative controller. The QP optimises generator voltages only; OLTC commands are handled separately by `action_oltc.m`. The two-step tap delay is managed by an explicit state correction rather than an augmented state. The Jacobian is recomputed before the power flow (after capacitor switching).

cal_mat_cst.m Builds constant MPC matrices without OLTC columns: Γ_{cst}, $f_{1,\text{cst}}$, $f_{2,\text{cst}}$, b_1, b_2, H_u, H_x, Ψ, $\tilde{C}$, A, C, J_1. Called once at initialisation.

Cal_mat_var.m Builds the operating-point-dependent matrices without OLTC columns: A_{ineq}, H, f_1, f_2, B. Called at every step.

action_oltc.m Enumeration-based OLTC decision logic. Evaluates tap-up, tap-down, and hold for each transformer at the predicted state $\hat{x}(k+2)$, selects the action minimising J_{OLTC} subject to eligibility conditions (cooldown, past and future voltage stability).

Naive controller/

main.m Main script for the naive controller. The QP optimises generator voltages only (same structure as `Alternative OLTC logic/`). OLTC taps are commanded by passing the voltage reference of each secondary bus directly to `run_pf_oltc_step`: the tap advances one step whenever $|V_{\text{secondary}} - V_{\text{ref}}| >$ `tol`, with no enumeration, no cost function, no cooldown, and no anticipation of the two-step tap delay. Serves as a baseline for comparison with the more sophisticated OLTC strategies.

cal_mat_cst.m Identical copy from `Alternative OLTC logic/`.

Cal_mat_var.m Identical copy from `Alternative OLTC logic/`.

Files shared by all controllers

case9xx_Bsh.m Network definition (bus data, branch data, generator data, load data). Not modified during simulation.

run_pf_oltc_step.m Runs the power flow (via `runpf`) and advances the OLTC tap by one step toward the target. The only interface with MATPOWER for power flow computation.

Cal_CvCq.m Computes C_v and C_q from a given Jacobian.

Cal_Cv_capCq_cap.m Computes $C_{v,\text{cap}}$ and $C_{q,\text{cap}}$ from a given Jacobian.

Cal_Cv_tapCq_tap.m Computes $C_{v,\text{tap}}$ and $C_{q,\text{tap}}$ from a given Jacobian.

add_cap.m Capacitor/reactor switching logic. Selects and applies at most one switching action per step.

Cal_criterion.m Computes performance indices J_V, J_Q, J_{VQ} for logging and display.

Sensitivity test/

Test.m Validates the sensitivity matrices by comparing the analytically computed C_v, C_q, $C_{v,\text{cap}}$, $C_{q,\text{cap}}$, $C_{v,\text{tap}}$, $C_{q,\text{tap}}$ against empirical finite-difference approximations obtained by perturbing each actuator and running a full nonlinear power flow. Displays element-wise relative errors.

Cal_CvCq.m, **Cal_Cv_capCq_cap.m**, **Cal_Cv_tapCq_tap.m** Local copies of the sensitivity functions under test.

case9xx_Bsh.m Local copy of the network definition.